Efficient Evaluation of Multivariate Polynomials over Structured Subsets of $\mathbb{F}_q^n$

Vaibhav Dixit, Santanu Sarkar, Fukang Liu, Willi Meier

Abstract

Efficient evaluation of a multivariate polynomial of degree d over a finite space is a central primitive in algebraic cryptanalysis, particularly in exhaustive search attacks against multivariate public-key cryptosystems (MPKCs). For the Boolean space $\mathbb{F}_2^n$, Bouillaguet et al. introduced the fast exhaustive search (FES) algorithm at CHES 2010. This line of work was further developed by Dinur at EUROCRYPT 2021 and Bouillaguet at TOMS 2024. Extending beyond the Boolean setting, Furue and Takagi proposed an algorithm at PQCrypto 2023 that generalizes FES to the finite-field space $\mathbb{F}_q^n$, where q is a prime number, achieving time complexity $\mathcal{O}\big(d \cdot q^n\big)$ with an initialization cost of $\binom{n+d}{d}^2$ and memory complexity $\mathcal{O}\big(\log(q \cdot n) \cdot n \cdot \binom{n+d}{d}\big)$. However, all these algorithms operate over the full space $\mathbb{F}_q^n$, which limits their applicability in many cryptanalytic scenarios where polynomial evaluation is required only over specific subsets of $\mathbb{F}_q^n$, such as those arising in the Syndrome Decoding Problem. Recently, Liu et al. proposed a memory-efficient algorithm for evaluating polynomials over the structured subset $P_{n_s}^{w_s} \times \cdots \times P_{n_1}^{w_1} \subseteq \mathbb{F}_2^n$, where $\sum_{i=1}^{s} n_i = n$ and $P_{n_i}^{w_i} \subseteq \mathbb{F}_2^{n_i}$ denotes the set of vectors of length n_i with Hamming weight at most w_i. In this work, we extend the structured-subset evaluation paradigm from the Boolean setting to arbitrary finite fields $\mathbb{F}_q$. Building on the abstraction of evaluation rules and evaluation orders introduced by Liu et al., and combining it with higher-order derivative techniques over finite fields, we develop a unified theoretical framework for evaluating multivariate polynomials over the structured subset S of $\mathbb{F}_q^n$. We derive two methods for the initialization phase: a coefficient-based approach using the coefficients of the polynomial and a derivative-based approach exploiting its higher-order derivatives. The former achieves time complexity $\mathcal{O}\big(d \cdot \binom{2n+d}{d}\big)$ with memory requirement $\mathcal{O}\big(\log q \cdot \binom{n+d}{d} + \log q \cdot d^2\big)$, while the latter runs in time $\mathcal{O}\big(\binom{n+d}{d}^2\big)$ and requires $\mathcal{O}\big(\log q \cdot \binom{n+d}{d}\big)$ memory. Depending on the values of n and d, the appropriate method is selected for initialization. After initialization, a degree-d polynomial can be evaluated over a structured subset S of $\mathbb{F}_q^n$ with time complexity $\mathcal{O}\big(d \cdot |S|\big)$.

Index Terms

Multivariate polynomials, Fast enumeration algorithms, polynomial evaluation, polynomial method, FES, MPKCs

I. Introduction

Evaluating multivariate polynomials over finite domains is a fundamental task in computer algebra and cryptography. Such evaluations arise naturally in fast exhaustive search and algebraic cryptanalysis [1], [2], and form a core computational primitive in attacks on multivariate public-key cryptosystems (MPKCs), which are among the leading candidates for post-quantum cryptography. In particular, enumeration problems, where a degree-d polynomial is evaluated over a large subset of $\mathbb{F}_q^n$ to identify its zeros, play a central role in XL-type and Crossbred attacks [3], [4]. While naive enumeration is computationally infeasible, the low algebraic degree of the target polynomials enables significantly faster evaluation strategies.

A notable example is the Fast Exhaustive Search (FES) technique introduced by Bouillaguet *et al.* for Boolean polynomials [5]. By traversing $\mathbb{F}_2^n$ in Gray code order and exploiting Boolean derivatives, FES evaluates a degree-d polynomial on all 2^n inputs in time $\mathcal{O}(d \cdot 2^n)$, following a preprocessing phase of cost $\binom{n}{\leq d}^2$ using $\binom{n}{\leq d}$ bits of memory. This approach outperforms the standard Möbius transform in time complexity and improves upon the algorithm of [6] in terms of memory usage. Subsequently, several algorithms were proposed [7]–[9] to solve systems of Boolean equations with improved complexities; however, these methods remain restricted to the Boolean space $\mathbb{F}_2^n$.

The limitation to Boolean polynomials was later partially overcome by extending FES to arbitrary finite fields. Furue and Takagi proposed an enumeration algorithm at PQCrypto 2023 [10], replacing Gray code traversal with lexicographic enumeration and maintaining higher-order derivatives. They showed that a degree-d polynomial can be evaluated over the full space $\mathbb{F}_q^n$ in time $O(d \cdot q^n)$, after a preprocessing phase requiring approximately

V. Dixit and S. Sarkar are with the Department of Mathematics, Indian Institute of Technology Madras, Chennai 600036, India (e-mail: vaibhavrdrk@gmail.com; sarkar.santanu.bir1@gmail.com).

F. Liu is with the Institute of Science Tokyo, Japan (e-mail: liufukangs@gmail.com).

W. Meier is with the University of Applied Sciences and Arts Northwestern Switzerland (e-mail: willimeier48@gmail.com).

$\binom{n+d}{d}^2$ arithmetic operations [10]. This improvement comes at the expense of increased memory complexity, namely $\mathcal{O}(\log(q \cdot n) \cdot n \cdot \binom{n+d}{d})$ bits. Nevertheless, this approach applies only to the full finite space $\mathbb{F}_q^n$ and does not exploit additional structure in the evaluation domain.

However, in many cryptanalytic applications, polynomials must be evaluated over structured subsets of $\mathbb{F}_q^n$. A prominent example is the Syndrome Decoding Problem (SDP), where a matrix $A \in \mathbb{F}_q^{(n-k)\times n}$, a syndrome $s \in \mathbb{F}_q^{n-k}$, and an integer weight $w \in \{0, \ldots, n\}$ are given, and the goal is to find a vector $x \in \mathbb{F}_q^n$ such that

$$Ax^\top = s^\top \quad \text{and} \quad Hw(x) \leq w,$$

where the Hamming weight of x is defined as

$$Hw(x) = \big|\{\, i \mid x_i \neq 0 \,\}\big|.$$

SDP is a classical problem in coding theory and is known to be NP-complete [11]. Recently, the signature scheme WAVE was proposed in [12], relying on the hardness of SDP over $\mathbb{F}_3^n$ with large weights. In such problems, the search space is not the full space $\mathbb{F}_q^n$, but rather a restricted subset of vectors with a Hamming weight not greater than w. We refer to this as a *structured subset* and denote it by $P_n^w \subseteq \mathbb{F}_q^n$. Similar structured search spaces also arise in [7], where systems of Boolean equations are solved over the structured subset of the form $P_{n-n_1}^w \times P_{n_1}^{n_1} \subseteq \mathbb{F}_2^n$.

Motivated by these settings, Liu *et al.* [13] revisited the fast exhaustive evaluation over structured subsets of $\mathbb{F}_2^n$ by decoupling the notion of an evaluation rule from the evaluation order. While this abstraction enables fast evaluation over a broad class of structured Boolean subsets, the technique remains inherently restricted to the Boolean field.

A. Our contribution

In this paper, we bridge this gap by extending efficient polynomial evaluation over a structured subset from the Boolean setting to arbitrary finite fields $\mathbb{F}_q^n$. By combining the evaluation-rule abstraction for structured subsets [13] with derivative-based update techniques over finite fields [10], we develop a unified framework for efficient polynomial evaluation over structured subsets of $\mathbb{F}_q^n$, where q is a prime number. We then extend the construction to the case where q is a prime power, i.e., $q = p^r$ for a prime p and a positive integer r.

B. Organization

In Section II, we introduce the notation, review existing polynomial evaluation algorithms over the entire space $\mathbb{F}_q^n$, and describe the evaluation framework over the structured subset of $\mathbb{F}_2^n$. In Section III, we extend this framework from the Boolean field to any finite field of prime order. We further show that the proposed algorithm extends naturally to finite fields of prime power order, that is, to $\mathbb{F}_{p^r}$ for $r > 1$ (Remark 5). In Section IV, we present the proposed algorithm and analyze its complexity. In Section V, we establish its correctness. In Section VI, we provide illustrative examples demonstrating the working of the proposed algorithm. Finally, in Section VII, we conclude the paper.

II. Preliminaries and Evaluation Framework

A. Notation

The following notations will be used throughout the paper.

- The bold character $\boldsymbol{x}$ is used to represent a vector of length n, i.e.,
$$\boldsymbol{x} = (x_n, \cdots, x_1).$$
- Algebra of vectors and functional values: If $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{F}_q^n$ are two vectors and $f(\boldsymbol{x})$, $f(\boldsymbol{y})$ are the corresponding values of the function f, then
$$\begin{aligned} \boldsymbol{x} \ \pm \ \boldsymbol{y} &= \big((x_n \pm y_n) \bmod q, \cdots, (x_1 \pm y_1) \bmod q\big)\ , \\ \boldsymbol{x} \ \times \ \boldsymbol{y} &= \big((x_n \times y_n) \bmod q, \cdots, (x_1 \times y_1) \bmod q\big)\ , \\ f(\boldsymbol{x}) \ \pm \ f(\boldsymbol{y}) &= \big(f(\boldsymbol{x}) \ \pm \ f(\boldsymbol{y})\big) \bmod q\ , \\ \text{for } \delta \in \mathbb{Z},\ \delta \cdot \boldsymbol{x} &= \big((\delta \cdot x_n) \bmod q, \cdots, (\delta \cdot x_1) \bmod q\big) \end{aligned}$$
- The sum over the entries of a vector is considered as an integer sum, i.e., for $\boldsymbol{x} \in \mathbb{F}_q^n$, $\sum x_i \in \mathbb{Z}$.
- The Hamming weight of the vector $\boldsymbol{x} \in \mathbb{F}_q^n$ is denoted by $\mathrm{Hw}(\boldsymbol{x})$ and is defined as the number of nonzero coordinates of $\boldsymbol{x}$, i.e.,
$$\mathrm{Hw}(\boldsymbol{x}) = \#\{i \mid x_i \neq 0\}.$$

- The truncated sum of the vector $\boldsymbol{x} \in \mathbb{F}_q^n$, for a given parameter ϵ, where $0 < \epsilon \leq n$, is denoted by $\mathrm{Tsum}_\epsilon(\boldsymbol{x})$ and is defined as
$$\mathrm{Tsum}_\epsilon(\boldsymbol{x}) = \sum_{i=1}^{\epsilon} x_i \quad \text{and } \mathrm{Tsum}_0(\boldsymbol{x}) = 0.$$
- We define the function $INT(\boldsymbol{x})$ as $INT(\boldsymbol{x}) = \sum_{i=1}^{n} q^{i-1} x_i$.
- $\boldsymbol{e}_i = (0,\ldots,0,1,\underbrace{0,\ldots,0}_{i-1})$ and $\overline{\boldsymbol{e}_i} = (1,\ldots,1,0,\underbrace{1,\ldots,1}_{i-1})$ where $1 \leq i \leq n$.
- For a vector $\boldsymbol{x} = (x_n,\ldots,x_1) \in \mathbb{F}_q^n$ and $\delta \in \mathbb{Z}$, we define
$$\begin{aligned}
\boldsymbol{x}_{(k_i^\delta)} &= \boldsymbol{x} - \delta \boldsymbol{e}_{k_i}, \text{for } 0 \leq \delta < \boldsymbol{x}_{k_i} \\
\boldsymbol{x}^{(k_i)} &= \boldsymbol{x} \times \overline{\boldsymbol{e}_{k_i}}, \text{for } 1 \leq k_i \leq n, \\
\boldsymbol{x}^{(0)} &= \boldsymbol{x},\ \boldsymbol{x}^{(0,k_1,\ldots,k_i)} = \boldsymbol{x} \times (\overline{\boldsymbol{e}_{k_i}} \times \cdots \times \overline{\boldsymbol{e}_{k_1}}), \\
\boldsymbol{x}^{(0,k_1,\ldots,k_i)}_{(k_j^{\delta_j},k_l^{\delta_l})} &= (\boldsymbol{x} - \delta_j \boldsymbol{e}_{k_j} - \delta_l \boldsymbol{e}_{k_l}) \times (\overline{\boldsymbol{e}_{k_i}} \times \cdots \times \overline{\boldsymbol{e}_{k_1}}).
\end{aligned}$$
Clearly, $\boldsymbol{x}_{(k_i^{\delta+1})} = \boldsymbol{x}^{(k_i)}$, for $\delta = x_{k_i} - 1$
- $\underbrace{a,\ldots,a}_{j}$ represents j repetitions of a , denoted by a^j.

B. Algorithms for Fast Polynomial Evaluation

Several algorithms have been proposed to evaluate a multivariate polynomial f with different trade-offs between time and memory.

- **Standard Möbius Transform (MT).** The standard MT evaluates f on all inputs using an array of size 2^n. It consists of an initialization phase that stores all coefficient values of f, followed by n update steps. The time complexity is $\mathcal{O}(n \cdot 2^n)$ and the memory complexity is 2^n bits. For a degree-d polynomial, Dinur and Shamir [6] showed that time complexity can be reduced to $\mathcal{O}(d \cdot 2^n)$, while memory complexity remains 2^n.
- **Fast Exhaustive Search (FES) [5].** Proposed by Bouillaguet *et al.*, this algorithm traverses $\mathbb{F}_2^n$ using the Gray code order and exploits derivatives of f up to order d. The time complexity is $\binom{n}{\leq d}^2 + d \cdot 2^n$ and the memory complexity is $\binom{n}{\leq d}$.
- **Memory-efficient Möbius Transform [7].** Dinur proposed a fix-and-update strategy in which $n - d$ variables are fixed, and the standard MT is applied to the resulting d-variable polynomial. Intermediate polynomials are stored to speed up updates. The memory complexity is less than $n \cdot \binom{n}{\leq d}$ and the time complexity is smaller than $n \cdot 2^n$ but larger than $d \cdot 2^n + 2^{n-1}$.
- **Improved Memory-efficient MT [7].** By fixing $n' = n - \log \binom{n}{\leq d}$ variables and avoiding the storage of intermediate polynomials, the memory complexity is reduced to $2 \cdot \binom{n}{\leq d}$, while the time complexity remains below $n \cdot 2^n$.
- **Optimized Memory-efficient MT [8].** Bouillaguet further optimized the polynomial update step after fixing $n - d$ variables. Memory complexity is $\binom{n}{\leq d}$ and time complexity is $\mathcal{O}(d \cdot 2^n)$.
- **Fast Enumeration Algorithm (FEA) [10].** Introduced by Furue and Takagi, this algorithm generalizes FES to a finite field $\mathbb{F}_q^n$ using lexicographic order. The time complexity and the memory complexity are $\binom{n+d}{d}^2 + d \cdot q^n$ and $\log(q \cdot n) \cdot n \cdot \binom{n+d}{d}$, respectively.
- **Polynomial Evaluation over the Structured Subset of $\mathbb{F}_2^n$ [13].** Liu *et al.* reformulated the FES framework in a more abstract manner by introducing a non-recursive evaluation scheme. They proposed an efficient algorithm to evaluate a Boolean polynomial f of degree d in n variables over a structured subset $P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1} \subseteq \mathbb{F}_2^n$.

Since our contribution extends the framework of [13] from the Boolean setting to general finite fields $\mathbb{F}_q$, for completeness and to facilitate later developments, we briefly recall the underlying idea.

C. Evaluating a Polynomial over Structured Subsets of $\mathbb{F}_2^n$

In [13], a structured subset of $\mathbb{F}_2^n$ is denoted by $P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1}$ and is defined as

$$\begin{aligned}
P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1} &= P_{n_s}^{w_s} \times \cdots \times P_{n_2}^{w_2} \times P_{n_1}^{w_1} \subseteq \mathbb{F}_2^n, \quad n = \sum_{i=1}^{s} n_i, \\
\forall i \in [1,s]: \quad P_{n_i}^{w_i} &= \{\boldsymbol{x} \in \mathbb{F}_2^{n_i} \mid \mathrm{Hw}(\boldsymbol{x}) \leq w_i\}.
\end{aligned}$$

To evaluate a polynomial f over a structured subset, the authors first derived a non-recursive form from the recursive relation used in the derivative-based approach [5], [10]. In the derivative-based approach, polynomial evaluation is naturally expressed by a recursive identity that relates the value of a polynomial to that of an appropriate derivative polynomial evaluated at a modified input. In particular, to evaluate a Boolean polynomial $f(x_n, \ldots, x_1)$ of degree d over the full space $\mathbb{F}_2^n$, the following recursive relation is used:

$$\forall i \in [1, n]: \quad f(\boldsymbol{x}) \quad = \quad f(\boldsymbol{x}_{(i)}) \oplus D_i(f)(\boldsymbol{x}^{(i)}). \tag{1}$$

All notation used here is defined in Section II-A, where subtraction is interpreted as $\oplus$ and multiplication as $\times$ in the Boolean setting.
Here, $D_i(f)$ represents the derivative of f with respect to x_i and is defined as

$$D_i(f)(\boldsymbol{x}) = f(\boldsymbol{x} \oplus e_i) \oplus f(\boldsymbol{x}).$$

Since $D_i(f)$ is again a polynomial of degree $d-1$, the relation in Equation 1 can be applied recursively by replacing the polynomial f and the input $\boldsymbol{x}$ with $D_i(f)$ and $\boldsymbol{x}^{(i)}$, respectively. Iterating this process reduces the evaluation to that of a constant polynomial. In this way, $f(x_n, \ldots, x_1)$ can be evaluated for any input in $\mathbb{F}_2^n$.

To evaluate $f(\boldsymbol{x})$, the recursive relation in Equation 1 can be reformulated into a non-recursive form. That is, there exists $\ell \in [1, d]$ and a set of indices $\{k_1, \ldots, k_\ell\} \subseteq \{1, \ldots, n\}$ and $k_0 = 0$ such that

$$\begin{cases} D_{k_0,\ldots,k_\ell}(f)(\boldsymbol{x}^{(k_0,\ldots,k_\ell)}) \text{ can be efficiently computed,} \\ D_{k_0,\ldots,k_{\ell-1}}(f)(\boldsymbol{x}^{(k_0,\ldots,k_{\ell-1})}) = D_{k_0,\ldots,k_{\ell-1}}(f)(\boldsymbol{x}^{(k_0,\ldots,k_{\ell-1})}_{(k_\ell)}) \oplus D_{k_0,\ldots,k_\ell}(f)(\boldsymbol{x}^{(k_0,\ldots,k_\ell)}), \\ \quad \vdots \\ D_{k_0,\ldots,k_i}(f)(\boldsymbol{x}^{(k_0,\ldots,k_i)}) = D_{k_0,\ldots,k_i}(f)(\boldsymbol{x}^{(k_0,\ldots,k_i)}_{(k_{i+1})}) \oplus D_{k_0,\ldots,k_{i+1}}(f)(\boldsymbol{x}^{(k_0,\ldots,k_{i+1})}), \\ \quad \vdots \\ D_{k_0}(f)(\boldsymbol{x}^{(k_0)}) = D_{k_0}(f)(\boldsymbol{x}^{(k_0)}_{(k_1)}) \oplus D_{k_0,k_1}(f)(\boldsymbol{x}^{(k_0,k_1)}). \end{cases} \tag{2}$$

Concretely, for a Boolean polynomial f of degree d, the non-recursive formulation shows that evaluating $f(\boldsymbol{x})$ can always be expressed as a finite chain of identities involving derivative polynomials of the form $D_{k_0,\ldots,k_i}(f)$ evaluated at modified inputs $\boldsymbol{x}^{(k_0,\ldots,k_i)}$. This chain terminates either when the derivative order reaches d or when the input becomes the zero vector. This reformulation plays a central conceptual role, as it explicitly identifies all derivative polynomials and inputs that may arise during evaluation and clarifies how values propagate from simpler to more complex instances.

Based on this non-recursive formulation, the authors introduced two key abstract notions, namely *evaluation rule* and *evaluation order*, together with a sufficient condition for efficient evaluation of f over any ordered space.

Evaluation Rule: Let $g(x_n, \ldots, x_1)$ be a polynomial evaluated over an ordered set S. An *evaluation rule* $\mathcal{R}$ assigns to every $\boldsymbol{x} \in S$ with $\boldsymbol{x} \neq S[0]$ a unique $\boldsymbol{x}' \in S$ such that

$$g(\boldsymbol{x}) = g(\boldsymbol{x}') \oplus D_i(g)(\boldsymbol{x}'), \quad \text{where } \boldsymbol{x} \oplus \boldsymbol{x}' = \boldsymbol{e}_i.$$

This relation is denoted by $\boldsymbol{x}' \xrightarrow{\mathcal{R}} \boldsymbol{x}$.

Evaluation Order: For a polynomial g evaluated over an ordered set S, we write $\boldsymbol{x}' <_g \boldsymbol{x}$ if the evaluation of $g(\boldsymbol{x}')$ is performed before that of $g(\boldsymbol{x})$.

If $\boldsymbol{x}' <_g \boldsymbol{x}$ and $\boldsymbol{x}' \xrightarrow{\mathcal{R}} \boldsymbol{x}$, we call $\boldsymbol{x}'$ a *predecessor* of $\boldsymbol{x}$ under the evaluation rule $\mathcal{R}$. These notions formalize how polynomial values propagate across the input set and allow the evaluation process to be described independently of any specific enumeration method. For convenience, throughout this paper, we use the notation

$$S_{k_0^{x_{k_0}},\ldots,k_{i-1}^{x_{k_{i-1}}},k_i^\delta}$$

to denote the ordered input set associated with the derivative

$$D_{k_0^{x_{k_0}},\ldots,k_{i-1}^{x_{k_{i-1}}},k_i^\delta}(f).$$

The corresponding evaluation rule is used to evaluate $D_{k_0^{x_{k_0}},\ldots,k_{i-1}^{x_{k_{i-1}}},k_i^\delta}(f)$ over the ordered set $S_{k_0^{x_{k_0}},\ldots,k_{i-1}^{x_{k_{i-1}}},k_i^\delta}$ is denoted by $\mathcal{R}_{k_0^{x_{k_0}},\ldots,k_{i-1}^{x_{k_{i-1}}},k_i^\delta}$.

In particular, when $x_{k_i} = 1$ for all i and $\delta = 0$, this notation coincides with the standard Boolean setting.

The non-recursive formulation further yields the following sufficient condition for a given evaluation order and update rule to make the above framework work efficiently.

Theorem 1 (Sufficient condition for derivative-based evaluation [13]). *Let $f(x_n, \ldots, x_1)$ be a polynomial of degree d evaluated over an ordered set S_0. Consider any triple $(g, S, \mathcal{R})$ belonging to*

$$\{(D_0(f), S_0, \mathcal{R}_0), \ldots, (D_{k_0,\ldots,k_{d-1}}(f), S_{k_0,\ldots,k_{d-1}}, \mathcal{R}_{k_0,\ldots,k_{d-1}})\},$$

where g is a polynomial arising during the non-recursive evaluation process, S is the ordered input set associated with g, and $\mathcal{R}$ is the corresponding evaluation rule.

If for every such $(g, S, \mathcal{R})$ and for all $\boldsymbol{x} \in S$ with $\boldsymbol{x} \neq S[0]$, there exists $\boldsymbol{x}' \in S$ such that

$$\boldsymbol{x}' <_g \boldsymbol{x} \quad \text{and} \quad \boldsymbol{x}' \xrightarrow{\mathcal{R}} \boldsymbol{x},$$

then the evaluation of f over S_0 can be carried out using at most $d \cdot |S_0|$ operations in the evaluation phase, in addition to the cost of computing all initial values $g(S[0])$ during the initialization phase.

Intuitively, this condition ensures that every evaluation step depends only on values computed earlier in the evaluation order and on at most one higher-order derivative. As a result, the total evaluation complexity is linear in $|S_0|$, up to a multiplicative factor d.

Using the lexicographic order as the evaluation order, the evaluation rule as defined in Equation 1, and the sufficient condition in Theorem 1, the authors of [13] showed that the evaluation of f over a structured subset S can be performed in $d \cdot |S| + \binom{n}{\leq d}$ operations, with a memory requirement of $2 \cdot \binom{n}{\leq d}$ bits. Based on the corresponding non-recursive relation, they further proposed an explicit algorithm (Algorithm 1 in [13]) for this evaluation technique. Since the evaluation of f in our setting relies fundamentally on this algorithmic framework, we next examine the key ideas underlying its construction.

In [13], the authors introduce several arrays of the form $A_{(k_0,\ldots,k_i)}$ for $0 \leq i \leq d$, which store the values of the intermediate polynomials arising in Equation 2. Since the evaluation of $g(\boldsymbol{x})$ depends on both the value of $g(\boldsymbol{x}')$ and a higher-order derivative, these quantities must be stored during computation. Storing all intermediate values simultaneously would incur a significant memory overhead and reduce the efficiency of the algorithm. To overcome this issue, the authors adopted memory-reuse strategy in which array entries no longer required for future evaluations are updated (i.e., overwritten) rather than permanently stored. The corresponding Memory allocation scheme and update rule are described below.

Memory allocation scheme: In the Boolean setting $\mathbb{F}_2$, the memory array $A_{(k_0,\ldots,k_i)}[\cdot]$ is used to store values of the derivative $D_{(k_0,\ldots,k_i)}f(\cdot)$. The length of this array is $n + 1 - k_i$. Initially, for each index $j > 0$, the entry $A_{(k_0,\ldots,k_i)}[j]$ stores

$$D_{(k_0,\ldots,k_i)}f\big(0,\ldots,0,1,\underbrace{0,\ldots,0}_{j-1},\underbrace{0,\ldots,0}_{k_i}\big).$$

As the evaluation proceeds, this memory is reused to store

$$D_{(k_0,\ldots,k_i)}f\big(\mathbf{a},1,\underbrace{0,\ldots,0}_{j-1},\underbrace{0,\ldots,0}_{k_i}\big), \quad \text{where } \mathbf{a} \in \mathbb{F}_2^{n-(j+k_i)}.$$

It was also proved that, once the value $D_{(k_0,\ldots,k_i)}f(\mathbf{a},1,\underbrace{0,\ldots,0}_{j-1},\underbrace{0,\ldots,0}_{k_i})$ have been computed, all previously stored values $D_{(k_0,\ldots,k_i)}f(\mathbf{a}',1,\underbrace{0,\ldots,0}_{j-1},\underbrace{0,\ldots,0}_{k_i})$ with

$$(\mathbf{a}',1,\underbrace{0,\ldots,0}_{j-1},\underbrace{0,\ldots,0}_{k_i}) <_{\text{lex}} (\mathbf{a},1,\underbrace{0,\ldots,0}_{j-1},\underbrace{0,\ldots,0}_{k_i})$$

become obsolete and can be safely released. This reuse-and-release strategy ensures that the algorithm remains memory efficient.

Example: Let $n = 3$ and consider the memory array $A_{(0)}$.

- $A_{(0)}[1]$ first stores $f(001)$. When this value is no longer required for subsequent computations, the same memory location is overwritten to store $f(011)$, then $f(101)$, and finally $f(111)$.
- $A_{(0)}[2]$ first stores $f(010)$ and is later overwritten to store $f(110)$.
- $A_{(0)}[3]$ stores $f(100)$, which does not need to be overwritten.

Similarly, for $A_{(1)}$, which stores the values of $D_1 f(\cdot)$, the same reuse principle applies:

- $A_{(1)}[1]$ first stores $D_1 f(010)$ and is subsequently overwritten by $D_1 f(110)$ and then $D_1 f(101)$.
- $A_{(1)}[2]$ stores $D_1 f(100)$.

Memory Update Rule: To evaluate $f(\boldsymbol{x})$, for a given value of $\boldsymbol{x}$, consider the i-th step of the non-recursive relation (Equation 2)

$$D_{k_0,\ldots,k_i}(f)(\boldsymbol{x}^{(k_0,\ldots,k_i)}) = D_{k_0,\ldots,k_i}(f)(\boldsymbol{x}^{(k_0,\ldots,k_i)}_{(k_{i+1})}) \oplus D_{k_0,\ldots,k_{i+1}}(f)(\boldsymbol{x}^{(k_0,\ldots,k_{i+1})})$$

If the input of $D_{k_0,\ldots,k_i}(f)$ is of the form

$$\boldsymbol{x}^{(k_0,\ldots,k_i)} = (\mathbf{a}, 1, \underbrace{0,\ldots,0}_{j-1}, \underbrace{0,\ldots,0}_{k_i}), \quad \text{where } \mathbf{a} \in \mathbb{F}_2^{n-k_i-j}.$$

Then the i-th step can be rewritten as

$$D_{(k_0,\ldots,k_i)} f\big(\mathbf{a}, 1, \underbrace{0,\ldots,0}_{j-1}, \underbrace{0,\ldots,0}_{k_i}\big) = D_{(k_0,\ldots,k_i)} f\big(\mathbf{a}, 0, \underbrace{0,\ldots,0}_{j-1}, \underbrace{0,\ldots,0}_{k_i}\big) \oplus D_{(k_0,\ldots,j+k_i)} f\big(\mathbf{a}, 0, \underbrace{0,\ldots,0}_{j-1}, \underbrace{0,\ldots,0}_{k_i}\big). \tag{3}$$

If $\mathbf{a} \neq \mathbf{0}$ and is of the form $\mathbf{a}'1\underbrace{0,\ldots,0}_{j'-1}$, the corresponding memory update rule is

$$A_{(k_0,\ldots,k_i)}[j] = A_{(k_0,\ldots,k_i)}[j'+j] \oplus A_{(k_0,\ldots,j+k_i)}[j'].$$

Since

$$\begin{aligned} j + k_i &= k_{i+1}, \\ j + j' + k_i &= k_{i+2}, \end{aligned} \qquad \Longrightarrow \qquad j = k_{i+1} - k_i, \quad j' = k_{i+2} - k_{i+1}.$$

Therefore, the update rule simplifies to

$$A_{(k_0,\ldots,k_i)}[k_{i+1} - k_i] = A_{(k_0,\ldots,k_i)}[k_{i+2} - k_i] \oplus A_{(k_0,\ldots,k_i,k_{i+1})}[k_{i+2} - k_{i+1}]. \tag{4}$$

The memory update rule at the termination step, i.e., either $\mathbf{a} = \mathbf{0}$ or $i = d-1$, can be derived from Equation 3 as follows.

1) Case ($\mathbf{a} = \mathbf{0}$): In this case, $i = h-1$, where $h = \mathrm{Hw}(\boldsymbol{x})$ and the right-hand side of Equation 3 reduces to
$$D_{(k_0,k_1,\ldots,k_{h-1})} f(\mathbf{0}) \oplus D_{(k_0,k_1,\ldots,k_{h-1},k_h)} f(\mathbf{0}),$$
which correspond to the sum of coefficients of $x^{k_1}\cdots x^{k_{h-1}}$ and $x^{k_1}\cdots x^{k_h}$ in f, stored in $A_{(k_0,k_1,\ldots,k_{h-1})}[0]$ and $A_{(k_0,k_1,\ldots,k_{h-1},k_h)}[0]$, respectively.
2) Case ($i = d-1$): In this case, the last term on the right-hand side of Equation 3 becomes
$$D_{(k_0,\ldots,k_d)} f(\mathbf{a}, 0, \underbrace{0,\ldots,0}_{j-1}, \underbrace{0,\ldots,0}_{k_i}) = D_{(k_0,\ldots,k_d)} f(\mathbf{0}),$$
since $D_{(k_0,\ldots,k_d)} f(\cdot)$ is independent of input. Consequently, the value of $D_{(k_0,\ldots,k_d)} f(\mathbf{0})$ can be obtained from $A_{(k_0,\ldots,k_d)}[0]$.

III. Extension to Finite Fields $\mathbb{F}_q$

In this work, we adopt the same abstract framework. We first derive an analogous non-recursive evaluation form for polynomials over $\mathbb{F}_q^n$ where q is a prime number, which identifies the derivative polynomials and intermediate inputs that arise during the evaluation. Using this formulation, we define evaluation rules and evaluation orders over $\mathbb{F}_q^n$ in direct analogy with the Boolean case. We then derive a sufficient condition for the applicability of this framework to $\mathbb{F}_q^n$. Finally, based on the evaluation rule, we derive a memory allocation and update rule that enables the construction of an algorithm to evaluate f over structured subsets of $\mathbb{F}_q^n$.

Consider a polynomial $f : \mathbb{F}_q^n \to \mathbb{F}_q$ in n variables of degree d, whose general form is

$$f(x_1,\ldots,x_n) = \sum_{\substack{\mathbf{i}\in\mathbb{N}^n \\ |\mathbf{i}|\le d,\ 0\le i_k\le q-1}} a_{\mathbf{i}} \prod_{k=1}^{n} x_k^{i_k} \tag{5}$$

where $\mathbb{N} = \{0, 1, \cdots\}$, $|\mathbf{i}| = \sum i_k$ and $a_{\mathbf{i}} \in \mathbb{F}_q$.

Definition 1. *(i) For $k \in \{1,\ldots,n\}$, the derivative of f with respect to the variable x_k is defined as*

$$D_k f(\boldsymbol{x}) \coloneqq f(\boldsymbol{x} + \boldsymbol{e}_k) - f(\boldsymbol{x})$$

where $\boldsymbol{x} \in \mathbb{F}_q^n$.

(ii) *For* $m \geq 2$, *the m-th order derivative of* f *with respect to* x_k *is as*

$$D_{k^m} f(\boldsymbol{x}) := D_{k^{m-1}} f(\boldsymbol{x} + \boldsymbol{e}_k) - D_{k^{m-1}} f(\boldsymbol{x}),$$

with $D_{k^1} f := D_k f$.

For distinct indices $k_1, \ldots, k_h \in \{1, \ldots, n\}$ *and integers* $m_1, \ldots, m_h > 0$, *we define the mixed higher–order derivative by*

$$D_{k_1^{m_1}, \ldots, k_h^{m_h}} f \;:=\; D_{k_h^{m_h}} \cdots D_{k_1^{m_1}} f.$$

Throughout the paper, when evaluating f at a fixed input $\boldsymbol{x} \in \mathbb{F}_q^n$, we use the shorthand notation

$$D_{k^{x_k}} f$$

to denote the x_k-th order derivative of f with respect to the variable x_k. In the notation, x_k refers to the integer value of the k-th coordinate of the fixed input $\boldsymbol{x}$, not to the formal indeterminate in the polynomial ring.

Although this slightly abuses notation, it will not cause ambiguity in the context of fixed input evaluation.

The fast enumeration algorithm [10] is based on the following recursive relation used to evaluate a polynomial $f(x_n, \ldots, x_1)$ of degree d in the full space $\mathbb{F}_q^n$:

$$\forall k \in [1, n] : \; f(\boldsymbol{x}) \quad = \quad f(\boldsymbol{x}_{(k)}) + D_k(f)(\boldsymbol{x}_{(k)}), \tag{6}$$

As $D_k(f)$ is again a polynomial of degree $d-1$, Equation 6 can be applied recursively by replacing the polynomial f and the input $\boldsymbol{x}$ with $D_k(f)$ and $\boldsymbol{x}_{(k)}$, respectively. Iterating this process reduces the evaluation to a constant polynomial, allowing $f(x_n, \ldots, x_1)$ to be computed for any $\boldsymbol{x} \in \mathbb{F}_q^n$. Let us convert this recursive relation into a non-recursive form to analyze all intermediate derivative polynomials obtained during the evaluation process.

Non-recursive form of Evaluation Framework: To evaluate $f(\boldsymbol{x})$, the recursive form in Equation 6 can be rephrased in the way that $\exists\ \epsilon \in \{1, \cdots, d\}$, $\{k_1, \ldots, k_\epsilon\} \subseteq \{1, \ldots, n\}$, $k_0 = 0$, and $0 \leq \delta < x_{k_\epsilon}$, such that

$$\begin{cases} D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}} k_\epsilon^{\delta+1}}(f)(\boldsymbol{x}^{(k_0, \ldots, k_{\epsilon-1})}_{(k_\epsilon^{\delta+1})}) \text{ can be efficiently computed} \\ D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}} k_\epsilon^{\delta}}(f)(\boldsymbol{x}^{(k_0, \ldots, k_{\epsilon-1})}_{k_\epsilon^{\delta}}) = D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}} k_\epsilon^{\delta}}(f)(\boldsymbol{x}^{(k_0, \ldots, k_{\epsilon-1})}_{(k_\epsilon^{\delta+1})}) + D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}} k_\epsilon^{\delta+1}}(f)(\boldsymbol{x}^{(k_0, \ldots, k_{\epsilon-1})}_{(k_\epsilon^{\delta+1})}) \\ \cdots \\ D_{k_0^{x_{k_0}}}(f)(\boldsymbol{x}^{(k_0)}) = D_{k_0^{x_{k_0}}}(f)(\boldsymbol{x}^{(k_0)}_{(k_1)}) + D_{k_0^{x_{k_0}}, k_1}(f)(\boldsymbol{x}^{(k_0)}_{(k_1)}) \end{cases} \tag{7}$$

Equation 7 provides a non-recursive description of the evaluation process, where the computation of $f(\boldsymbol{x})$ is reduced to a sequence of derivative evaluations.

Terminating conditions: The recursion terminates in one of the following cases:

Case 1: If $\sum_{i=1}^{\epsilon-1} x_{k_i} + \delta + 1 = d$. In this case,

$$\forall \boldsymbol{x} \in \mathbb{F}_q^n : D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}}, k_\epsilon^{\delta+1}}(f)\left(\boldsymbol{x}^{(k_0, \ldots, k_{\epsilon-1})}_{(k_\epsilon^{\delta+1})}\right) = D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}}, k_\epsilon^{\delta+1}}(f)(\boldsymbol{0}^n).$$

Case 2: If $\boldsymbol{x}^{(k_0, \ldots, k_{\epsilon-1})}_{(k_\epsilon^{\delta+1})} = \boldsymbol{0}^n$. In this case, $\delta + 1 = x_{k_\epsilon}$, i.e., $\boldsymbol{x}^{(k_0, \ldots, k_{\epsilon-1})}_{(k_\epsilon^{\delta+1})} = \boldsymbol{x}^{(k_0, \ldots, k_{\epsilon-1}, k_\epsilon)} = \boldsymbol{0}^n$. Thus, the required value is

$$D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}}, k_\epsilon^{x_{k_\epsilon}}}(f)(\boldsymbol{0}^n).$$

From the termination conditions, it follows that at most d steps are required to evaluate $f(\boldsymbol{x})$ using Equation 7, with the final step yielding the value of $f(\boldsymbol{x})$.

Requirements of the Evaluation process: Due to the two terminating conditions of the above framework, the values of certain derivative polynomials evaluated at the zero vector must be precomputed and stored. In particular, the derivatives of the form

$$D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}}, k_\epsilon^{\delta+1}}(f)$$

where $1 \leq k_1 < \ldots < k_\epsilon \leq n$, $x_{k_i} \in \mathbb{F}_q \ \forall i > 0$, $0 \leq \delta < x_{k_\epsilon}$, and $0 \leq \sum_{i=1}^{\epsilon-1} x_{k_i} + \delta + 1 \leq d$ may occur at a terminal step. Therefore, the values of

$$D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}}, k_\epsilon^{\delta+1}}(f)(\boldsymbol{0}^n)$$

must be available. This guarantees that, whenever the evaluation procedure terminates, the required value can be retrieved directly without further computation.

Formally, the initialization phase of polynomial evaluation requires computing and storing the following set:

$$S_{\text{ini}} = \left\{ D_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}(f)(\mathbf{0}^n) \right\},$$

where the parameters satisfy

$$\begin{aligned} &1 \le k_1 < \cdots < k_\epsilon \le n, \\ &x_{k_i} \in \mathbb{F}_q \quad \text{for all } i > 0, \\ &0 \le \delta < x_{k_\epsilon}, \\ &0 \le \sum_{i=1}^{\epsilon-1} x_{k_i} + \delta + 1 \le d. \end{aligned}$$

Since our evaluation framework is based on a non-recursive formulation involving generalized derivatives of the form $D_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}k_\epsilon^{\delta+1}}(f)$, the sufficient condition ensuring efficient evaluation must be adapted accordingly.

Theorem 2 (Sufficient condition). *Let $f(x_n,\ldots,x_1)$ be a polynomial of degree d evaluated over an ordered input set S_0 according to Equation 7. For any non-constant polynomial g appearing in Equation 7, let S be the ordered input set associated with g, and let $\mathcal{R}$ denote the evaluation rule used to evaluate g over S. Then*

$$(g,S,\mathcal{R}) \in \left\{ (D_0(f),S_0,\mathcal{R}_0), (D_{k_0^{x_{k_0}},k_1}, S_{k_0^{x_{k_0}},k_1}, \mathcal{R}_{k_0^{x_{k_0}},k_1}), \ldots, (D_k(f),S_k,\mathcal{R}_k) \right\},$$

where $k = k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}$ satisfies $0 \le \sum_{i=1}^{\epsilon-1} x_{k_i} + \delta + 1 < d$. If for every such triple $(g,S,\mathcal{R})$, the condition

$$\forall \boldsymbol{x} \in S,\ \boldsymbol{x} \neq S[0],\ \exists\, \boldsymbol{x}' \in S \text{ such that } \boldsymbol{x}' <_g \boldsymbol{x} \quad \text{and} \quad \boldsymbol{x}' \xrightarrow{\mathcal{R}} \boldsymbol{x} \tag{8}$$

holds, then evaluating f over S_0 requires at most $d \cdot |S_0|$ operations, in addition to the cost of computing all initial values $g(S[0])$ during the initialization phase.

Proof. To evaluate a polynomial over an ordered input set S_0 using Equation 7, we present an algorithm consisting of two phases: the initialization phase and the evaluation phase.

As discussed earlier, we first compute the values of the polynomial on the set S_{ini} and store them in a table $TAB_{\textbf{ini}}$, which constitutes the initialization phase of the algorithm. During the evaluation phase, we maintain a table $TAB_{\textbf{eva}}$ that stores all previously computed pairs $(\boldsymbol{x}, f(\boldsymbol{x}))$, and which is progressively updated as new values are computed. Enumerate $\boldsymbol{x} \in S_0$ in the evaluation order as defined over S_0, the procedure to evaluate $f(\boldsymbol{x})$ for each $\boldsymbol{x} \in S_0$ is executed as follows:

Step 1: If $\boldsymbol{x} = S_0[0]$, directly obtain $f(\boldsymbol{x})$ from the table $TAB_{\textbf{ini}}$. Add the pair $(\boldsymbol{x}, f(\boldsymbol{x}))$ to the table $TAB_{\textbf{eva}}$ for further use.

Step 2: If $\boldsymbol{x} = S_0[i]$ for $i > 0$, then according to the condition in Equation 8, there exists a predecessor $\boldsymbol{x}' = \boldsymbol{x}_{(k_1)}^{(k_0)}$ such that

$$f(\boldsymbol{x}) = D_{k_0^{x_{k_0}}}(f)\left(\boldsymbol{x}^{(k_0)}\right) = D_{k_0^{x_{k_0}}}(f)\left(\boldsymbol{x}_{(k_1)}^{(k_0)}\right) + D_{k_0^{x_{k_0}},k_1}(f)\left(\boldsymbol{x}_{(k_1)}^{(k_0)}\right).$$

Here, $D_{k_0^{x_{k_0}}}(f)(\boldsymbol{x}_{(k_1)}^{(k_0)}) = f(\boldsymbol{x}_{(k_1)}^{(k_0)})$ can be obtained from the table $TAB_{\textbf{eva}}$. Consequently, the problem reduces to computing $D_{k_0^{x_{k_0}},k_1}(f)(\boldsymbol{x}_{(k_1)}^{(k_0)})$.

To evaluate $D_{k_0^{x_{k_0}},k_1}(f)(\boldsymbol{x}_{(k_1)}^{(k_0)})$, move to Step 1 by considering $(D_0,S_0,\mathcal{R}_0)$ as $(D_{k_0^{x_{k_0}},k_1}, S_{k_0^{x_{k_0}},k_1}, \mathcal{R}_{k_0^{x_{k_0}},k_1})$ and replacing $\boldsymbol{x}$ with $\boldsymbol{x}_{(k_1)}^{(k_0)}$.

In general, Step 2 outputs a triplet of the form $(D_k(f),S_k,\mathcal{R}_k)$ together with $\boldsymbol{x}_{(k_\epsilon^{\delta+1})}^{(k_0,\ldots,k_{\epsilon-1})}$, where $k = k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}$.

The execution must terminate at some point, i.e., it begins backtracking when $k_0^{x_{k_0}},k_1^{x_{k_1}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}$ satisfies one of the following conditions:

$$\boldsymbol{x}_{(k_\epsilon^{\delta+1})}^{(k_0,\ldots,k_{\epsilon-1})} = \mathbf{0}^n \quad \text{or} \quad \sum_{i=1}^{\epsilon-1} x_{k_i} + \delta + 1 = d.$$

These cases correspond to the terminating conditions. At this stage, the value of the $D_k f(\boldsymbol{x}_{(k_\epsilon^{\delta+1})}^{(k_0,\ldots,k_{\epsilon-1})})$ is obtained from $TAB_{\textbf{ini}}$, i.e., $\boldsymbol{x}_{(k_\epsilon^{\delta+1})}^{(k_0,\ldots,k_{\epsilon-1})} = S_k[0]$. The algorithm then backtracks and successively computes the intermediate derivative values and stores them in $TAB_{\textbf{eva}}$, until $f(\boldsymbol{x})$ is determined and inserted into $TAB_{\textbf{eva}}$.

Based on the above procedure, computing $f(\boldsymbol{x})$ for any $\boldsymbol{x} \in S_0$ requires at most d operations. Consequently, evaluating f over S_0 takes at most $d \cdot |S_0|$ operations in the evaluation phase. Furthermore, the memory requirement is determined by the size of $TAB_{\mathbf{eva}}$, which is at most $d \cdot |S_0|$. □

IV. Algorithmic Framework and Complexity Analysis

We consider the problem of evaluating a polynomial $f(x_n, \ldots, x_1)$ of degree d on all inputs belonging to the structured subset $P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$, defined as

$$\begin{aligned} P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1} &= P^{w_s}_{n_s} \times \cdots \times P^{w_1}_{n_1} \subseteq \mathbb{F}_q^n, \qquad n = \sum_{i=1}^{s} n_i, \\ P^{w_i}_{n_i} &= \{\boldsymbol{x} \in \mathbb{F}_q^{n_i} \mid \mathrm{Hw}(\boldsymbol{x}) \le w_i\}, \qquad \forall i \in \{1, \ldots, s\}. \end{aligned}$$

Before proceeding to a detailed analysis of evaluation over structured subsets, we present our algorithm. Similar to other polynomial evaluation algorithms [5], [10], [13], our algorithm is also executed in two phases: the *initialization phase* and the *Evaluation phase*. The goal of the initialization phase is to compute and store the values of all derivatives appearing in the set S_{ini}. For this purpose, we construct several arrays, whose structures are described below. To facilitate the description of the algorithm, we also introduce the memory allocation and corresponding memory update rules, which determine how array entries are managed throughout the computation. The evaluation phase then applies Equation 7 together with this update mechanism to evaluate the polynomial over the structured subset.

A. Initialization Phase

First, we prepare $\binom{n+d}{d}$ arrays, classified into $d+1$ different types.

- Type-0: The array is denoted by A_0. The size of A_0 is $n+1$.
- Type-i $(0 < i < d)$: The array is denoted by $A_{0,k_1^{x_{k_1}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}$ and has size $n+2-k_\epsilon$, where $\epsilon > 0$, $1 \le k_1 < \cdots < k_\epsilon \le n$, $x_{k_i} \in \mathbb{F}_q$ for all $i > 0$, $0 \le \delta < x_{k_\epsilon}$, and $\sum_{i=1}^{\epsilon-1} x_{k_i} + \delta + 1 = i$.
- Type-d: The array has size 1 and is denoted by $A_{0,k_1^{x_{k_1}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}$, where $\epsilon > 0$, $1 \le k_1 < \cdots < k_\epsilon \le n$, $x_{k_i} \in \mathbb{F}_q$ for all $i > 0$, $0 \le \delta < x_{k_\epsilon}$, and $\sum_{i=1}^{\epsilon-1} x_{k_i} + \delta + 1 = d$.

Remark 1. *(i) The array A_0 is indexed starting from 1, i.e., $A_0[1], \ldots, A_0[n+1]$. All other arrays are indexed starting from 0.*

(ii) Throughout this paper, we set $k_0 = 0$ for notational convenience. Accordingly, all arrays can be written uniformly as $A_{k_0^{x_{k_0}},k_1^{x_{k_1}},\ldots,k_\epsilon^{\delta+1}}$.

Each array is initialized by computing its first entry, which corresponds precisely to an element of the set S_{ini}. Therefore, the initialization phase reduces to efficiently computing the values of higher-order derivatives of a polynomial f evaluated at 0^n.

To compute these derivatives, we derive two methods, which are described below.

1) Method 1 (Direct coefficient-based computation): In the first method, the required derivatives are computed directly from the coefficients of the polynomial f. This approach relies on a known result on higher-order discrete derivatives of monomials, which we recall next.

For a monomial $t = x_{s_1}^{m_1} \cdots x_{s_k}^{m_k}$, we use the shorthand notation

$$D_t := D_{s_1^{m_1},\ldots,s_k^{m_k}},$$

in accordance with Definition 1.

Theorem 3 (Higher-order discrete derivative of a monomial [14])**.** *Let $f : \mathbb{F}_q^n \to \mathbb{F}_q$.*

Suppose

$$f(x_1, \ldots, x_n) = x_{s_1}^{i_1} \cdots x_{s_k}^{i_k}, \qquad 1 \le m_\ell \le i_\ell \le q-1 \text{ for } 1 \le \ell \le k$$

and let $\boldsymbol{t} = x_{s_1}^{m_1} \cdots x_{s_k}^{m_k}$. Then

$$D_{\boldsymbol{t}} f(x) = \prod_{\ell=1}^{k} m_\ell! \sum_{j_1=m_1}^{i_1} \cdots \sum_{j_k=m_k}^{i_k} \Big(\prod_{\ell=1}^{k} \binom{i_\ell}{j_\ell} S(j_\ell, m_\ell) \Big) x_{s_1}^{i_1-j_1} \cdots x_{s_k}^{i_k-j_k}.$$

where $S(\cdot,\cdot)$ denotes the Stirling numbers of the second kind.

In particular, evaluating the derivative $D_t f$ at 0^n yields a closed-form expression

$$D_{\boldsymbol{t}} \left(\prod_{\ell=1}^{k} x_{s_\ell}^{i_\ell} \right) \Big|_{0^n} = \prod_{\ell=1}^{k} \big(m_\ell ! \, S(i_\ell, m_\ell) \big).$$

Remark 2. *Some basic properties of Stirling numbers of the second kind are*

- $S(i,i) = 1$ *for all* $i \geq 0$.
- $S(i,0) = 0$ *for all* $i > 0$.
- $S(i,1) = 1$ *for all* $i > 0$.

Using the linearity of the derivative operator, this result immediately extends from monomials to the general polynomial of the form as given in Equation 5. Consequently, with the convention that $S(i_k, m_k) = 0$ if $i_k < m_k$, the value of the derivative of f with respect to a monomial $\boldsymbol{m} = x_1^{m_1} \cdots x_n^{m_n}$ at 0^n is given by

$$D_{\boldsymbol{m}} f(0^n) = \sum_{\mathbf{i}} a_{\mathbf{i}} \prod_{k=1}^{n} \big(m_k ! \, S(i_k, m_k) \big) \tag{9}$$

Complexity analysis of the Initialization phase:

Time complexity: First, we analyze the computational cost of evaluating these derivatives by counting the arithmetic operations required.

a) Number of Multiplication Operations per Monomial.: Consider a monomial $\boldsymbol{m} = \prod_{j=1}^{n} x_j^{m_j}$ with $m_j \geq 0$ and $\sum_{j=1}^{n} m_j \leq d$. When evaluating $\prod_{k=1}^{n} \big(m_k ! \, S(i_k, m_k) \big)$ three cases may occur:

1) If $m_j = 0$ for some j and $i_j \neq 0$, then $S(i_j, 0) = 0$, and hence the entire product vanishes. Thus, the coefficients corresponding to monomials that contain variables not present in $\boldsymbol{m}$ do not contribute to $D_{\boldsymbol{m}} f(0^n)$.
2) If $m_j = 0$ for some j and $i_j = 0$, then $m_j ! S(i_j, m_j) = 1$, and this term can be ignored when counting multiplication operations
3) If $m_j \neq 0$ for some j and $i_j \geq m_j$, then the corresponding term contributes non-trivially to the product. Since $\sum_{j=1}^{n} m_j \leq d$, there are at most d nonzero values of m_j.

Assuming that all values $S(i_k, m_k)$ are precomputed and stored, evaluating $\prod_{k=1}^{n} \big(m_k ! \, S(i_k, m_k) \big)$ requires at most d multiplication operations. One additional multiplication is needed to multiply the resulting product by the coefficient $a_{\mathbf{i}}$. Hence, the total number of multiplication operations required per monomial is bounded by $d + 1$.

Remark 3. *All Stirling numbers of the second kind* $S(i_k, m_k)$ *for* $0 \leq m_k \leq i_k \leq d$ *can be precomputed and stored using* $\mathcal{O}(d^2)$ *memory. These values satisfy the recurrence relation*

$$S(i_k, m_k) = m_k \, S(i_k - 1, m_k) + S(i_k - 1, m_k - 1).$$

For a fixed i_k*, the value* $S(i_k, m_k)$ *can be computed using a constant number of arithmetic operations, as the values* $S(i_k - 1, m_k)$ *and* $S(i_k - 1, m_k - 1)$ *are computed and stored beforehand. Thus, computing all values for a fixed* i_k *requires* $\mathcal{O}(i_k)$ *operations. Summing over* $i_k = 0, 1, \ldots, d$*, the total time complexity for computing all required Stirling numbers of the second kind is* $\mathcal{O}(d^2)$.

b) Number of addition operations.: For a given monomial $\boldsymbol{m}$, define the set

$$S_{\boldsymbol{m}} = \left\{ \prod_{k=1}^{n} x_k^{i_k} \;:\; i_k \geq m_k \geq 0, \; \sum_{k=1}^{n} i_k \leq d \right\}.$$

The cardinality $|S_{\boldsymbol{m}}|$ equals the number of monomials contributing to $D_{\boldsymbol{m}} f(0^n)$, and hence also the number of addition operations required to compute this derivative. It follows that

$$|S_{\boldsymbol{m}}| \leq \binom{n + d - \sum_{k=1}^{n} m_k}{n}.$$

Note that $\binom{n+d-\sum_{k=1}^{n} m_k}{n}$ also includes monomials that do not contribute to the evaluation of $D_{\boldsymbol{m}} f(0^n)$. A tighter bound on $|S_{\boldsymbol{m}}|$ is provided in Appendix A.

Summing over all the monomials $\boldsymbol{m}$ with $0 < \sum m_i \leq d$, the total number of operations required in the initialization phase is bounded by

$$(d+1) \cdot \sum_{0 < \sum m_i \leq d} |S_{\boldsymbol{m}}| = (d+1) \cdot \sum_{M=1}^{d} \binom{n + d - M}{n} \cdot \#\{\boldsymbol{m} : \sum_{i=1}^{n} m_i = M\}.$$

The number of monomials $\boldsymbol{m}$ satisfying $\sum_{i=1}^{n} m_i = M$ is

$$\#\{\boldsymbol{m} : \sum_{i=1}^{n} m_i = M\} = \binom{M+n-1}{n-1}.$$

Hence,

$$\begin{aligned}(d+1)\cdot \sum_{0<\sum m_i \le d} |S_{\boldsymbol{m}}| &= (d+1)\cdot \sum_{M=1}^{d} \binom{n+d-M}{n}\binom{M+n-1}{n-1}. \\ &\le (d+1)\cdot \binom{2n+d}{2n}.\end{aligned}$$

Thus, the time complexity of the initialization phase is

$$\mathcal{O}\left(d\cdot\binom{2n+d}{d} + d^2\right) \sim \mathcal{O}\left(d\cdot\binom{2n+d}{d}\right). \tag{10a}$$

Memory complexity: During the initialization phase, there are $\binom{n+i-1}{i}$ different Type-i arrays for $0 \le i \le d$. In particular,

- The number of Type-0 arrays is 1, and its size is $n+1$.
- For $1 \le i < d$, the total number of Type-i arrays is $\binom{n+i-1}{i}$. Among these, $\binom{k_\epsilon+i-2}{i-1}$ arrays have size $n+2-k_\epsilon$ for $0 < k_\epsilon \le n$. Therefore, the total memory required for all Type-i arrays is

$$\sum_{k_\epsilon=1}^{n} (n+2-k_\epsilon)\binom{k_\epsilon+i-2}{i-1} = \binom{n+i-1}{i}\frac{n+1+2i}{i+1}.$$

- The total number of Type-d arrays is $\binom{n+d-1}{d}$, and the size of each array is 1.

Each memory location stores an element of $\mathbb{F}_q$, requiring $\log q$ bits. Therefore, the memory required to store the values of the polynomial obtained during the evaluation phase is

$$\log q \cdot \left[\sum_{i=0}^{d-1} M(i) + \binom{n+d-1}{d}\right] = \log q \cdot \left[2\binom{n+d}{d} - 1\right],$$

where

$$M(i) = \binom{n+i-1}{i}\frac{n+1+2i}{i+1} \text{ for } i \ge 0.$$

Therefore, the memory complexity of the initialization phase is

$$\mathcal{O}\left(\log q \cdot \binom{n+d}{d} + \log q \cdot d^2\right).^{1} \tag{10b}$$

Moreover, for $n \ge 3$, term $\binom{n+d}{d}$ dominates over d^2. Thus, the memory complexity for large values of n is $\mathcal{O}\left(\log q \cdot \binom{n+d}{d}\right)$.

2) Method 2 (Based on the expression of derivatives): In this method, we derive explicit expressions for the derivatives of f, that is, for $D_{\boldsymbol{m}}f$ corresponding to each monomial $\boldsymbol{m}$ of total degree at most d. The constant term of $D_{\boldsymbol{m}}f$ represents the value of $D_{\boldsymbol{m}}f(0^n)$. An explicit algorithm for the initialization phase using this approach is given in Algorithm 1. To find the values of $D_{\boldsymbol{m}}f(0^n)$, start Algorithm 1 with $\mathbf{a} = 0$.

Complexity analysis of the Initialization phase:

Time complexity: In this method, all derivatives of order at most d are computed. The number of such derivatives is $\binom{n+d}{d}$, and the cost of computing each derivative is at most $\binom{n+d}{d}$. Therefore, the time complexity of the initialization phase is

$$\mathcal{O}\left(\binom{n+d}{d}^2\right). \tag{11a}$$

Memory complexity: As in the first method, all Type-i arrays are prepared, which together require $\log q \cdot \left[2\binom{n+d}{d} - 1\right]$ bits of memory. In addition, polynomials of degrees ranging from d down to 0 must be stored simultaneously,

[1]The required values of $S(i_k, m_k)$ are computed modulo q. Thus, each one requires only $\log q$ memory.

since the input of INIT in Algorithm 1 is $D_k f$. The memory required for this additional storage is

$$\log q \cdot \sum_{i=0}^{d} \binom{n+i}{i} = \mathcal{O}\left(\log q \cdot \binom{n+d}{d}\right).$$

Hence, the total memory complexity of the initialization phase is

$$\mathcal{O}\left(\log q \cdot \binom{n+d}{d}\right). \tag{11b}$$

From Equations (10b) and (11b), it is clear that Method 2 is more efficient in terms of memory complexity. However, with respect to time complexity, a comparison of Equations (10a) and (11a) shows that Method 2 outperforms Method 1 only for parameter pairs $(n = 1, d \geq 2)$, $(n = 2, d \geq 3)$, $(n = 3, d \geq 11)$, and $(n = 4, d \geq 53)$. For all other parameter choices, Method 1 has a lower time complexity.

Before describing the evaluation phase, we introduce a definition and discuss the memory allocation process based on Equation 7.

Algorithm 1 Initialize the first element of each Type-i array for $0 \leq i \leq d$

```
1: procedure INIT(A, f, a)
2:     A[a].1 ← f(0)                ▷ A[(0)].1 = A[(0)][1],  A[a ≠ (0)].1 = A[a][0]
3:     if Tsum_n(a) = d then
4:         return A
5:     else
6:         s ← last element of a    ▷ s = 1 if a = (0)
7:         for k = s to n do
8:             A ← INIT(A, D_k(f), a ∪ {k})
9:         end for
10:        return A
11:    end if
12: end procedure
```

Definition 2. *The* support *of a vector* $(x_n, \ldots, x_1)$*, denoted by*

$$(k_h, \ldots, k_1) = Sup(x_n, \ldots, x_1) \qquad \text{with } k_h > \cdots > k_1,$$

is defined as the set of nonzero positions counted from x_1 *to* x_n*. Specifically,*

$$\begin{aligned} x_i &\neq 0, \quad \forall i \in \{k_h, \ldots, k_1\}, \\ x_j &= 0, \quad \forall j \in \{1, \ldots, n\} \setminus \{k_h, \ldots, k_1\}. \end{aligned}$$

In particular,

$$Sup(0, \ldots, 0) = 0.$$

B. Evaluation Phase

To efficiently evaluate $f(\boldsymbol{x})$ for an input $\boldsymbol{x}$ whose support is $(k_h, \ldots, k_\epsilon, \ldots, k_1)$, we adopt the evaluation process described in Equation 7. The arrays prepared during the initialization phase are used to store the values of the polynomials that arise throughout the evaluation process. We now describe the memory allocation scheme for these values, together with the corresponding memory update rule.

Memory allocation scheme: During the evaluation process, a derivative of the form $D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}}, k_\epsilon^\delta}(f)$ appears at a certain stage, with input of the form $(\mathbf{a}, x_{k_\epsilon} - \delta, \underbrace{0, \ldots, 0}_{j-1}, \underbrace{0, \ldots, 0}_{K-1})$. Accordingly, this stage of the evaluation can be written as

$$\begin{aligned} D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}}, k_\epsilon^\delta}(f)(\mathbf{a}, \zeta, \underbrace{0, \ldots, 0}_{j-1}, \underbrace{0, \ldots, 0}_{K-1}) &= D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}}, k_\epsilon^\delta}(f)(\mathbf{a}, \zeta - 1, \underbrace{0, \ldots, 0}_{j-1}, \underbrace{0, \ldots, 0}_{K-1}) \\ &\quad + D_{k_0^{x_{k_0}}, \ldots, k_{\epsilon-1}^{x_{k_{\epsilon-1}}}, k_\epsilon^{\delta+1}}(f)(\mathbf{a}, \zeta - 1, \underbrace{0, \ldots, 0}_{j-1}, \underbrace{0, \ldots, 0}_{K-1}) \end{aligned} \tag{12}$$

where $\mathbf{a} \in \mathbb{F}_q^{n-(j+K-1)}$, $\zeta = x_{k_\epsilon} - \delta > 0$, and

$$K = k_{\epsilon-1} + (k_\epsilon - k_{\epsilon-1})\,\mathrm{sgn}(\delta) = \begin{cases} k_{\epsilon-1}, & \delta = 0, \\ k_\epsilon, & \delta > 0. \end{cases}$$

The array $A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^\delta}[\cdot]$ is used to store the values of the polynomial $D_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^\delta} f(\cdot)$. Initially, for each index $j > 0$, the entry $A_{k_0^{x_{k_0}},k_1^{x_{k_1}}\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^\delta}[j]$ stores

$$D_{k_0^{x_{k_0}},k_1^{x_{k_1}}\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^\delta} f(0,\ldots,0,1,\underbrace{0,\ldots,0}_{j-1},\underbrace{0,\ldots,0}_{K-1}),$$

As the evaluation proceeds, this memory is reused to store

$$D_{k_0^{x_{k_0}},k_1^{x_{k_1}}\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^\delta} f(\mathbf{a},\zeta,\underbrace{0,\ldots,0}_{j-1},\underbrace{0,\ldots,0}_{K-1}),$$

for consecutive values of ζ with $1 < \zeta \le q-1$. This reuse strategy allows memory locations storing values that will no longer be referenced in subsequent evaluations to be released and overwritten with new values. Consequently, the same memory is repeatedly recycled throughout the evaluation process, making the algorithm memory-efficient. This property is formalized in Lemma 5.

Memory Update Rule: From Equation 12, four distinct memory update rules arise, depending on the values of δ and ζ, as described below.

- **Case 1:** $\delta > 0$. In this case, $K = k_\epsilon$ and $j = 1$.
  - **Rule 1:** If $\zeta = (x_{k_\epsilon} - \delta) > 1$,
  $$A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[1] + A_{k_0^{x_{k_0}},\ldots,k_\epsilon^{\delta+1}}[1].$$
  - **Rule 2:** If $\zeta = (x_{k_\epsilon} - \delta) = 1$,
  $$A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[k_{\epsilon+1} - k_\epsilon + 1] + A_{k_0^{x_{k_0}},\ldots,k_\epsilon^{\delta+1}}[k_{\epsilon+1} - k_\epsilon + 1].$$
- **Case 2:** $\delta = 0$. In this case, $K = k_{\epsilon-1}$ and $j > 1$.
  - **Rule 3:** If $\zeta = x_{k_\epsilon} > 1$,
  $$A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[k_\epsilon - k_{\epsilon-1} + 1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[k_\epsilon - k_{\epsilon-1} + 1] + A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}k_\epsilon}[1].$$
  - **Rule 4:** If $\zeta = x_{k_\epsilon} = 1$,
  $$A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[k_\epsilon - k_{\epsilon-1} + 1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[k_{\epsilon+1} - k_{\epsilon-1} + 1] + A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}k_\epsilon}[k_{\epsilon+1} - k_\epsilon + 1].$$

Based on the above update rules, an explicit algorithm for evaluating the polynomial at a given input $\boldsymbol{x} \in \mathbb{F}_q^n$ is presented in Algorithm 2.

Remark 4. *In the Boolean case, i.e., over $\mathbb{F}_2^n$, we have $\delta = 0$ and $x_{k_i} = 1$ for all $i \in \{0,\ldots,\epsilon\}$. Under these settings,* ***Rule 4*** *reduces (up to index relabeling, identifying $\epsilon - 1$ with i) to the update rule given in Equation 4, where the addition is over $\mathbb{F}_2$.*

To evaluate f over the structured subset $P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1}$ of $\mathbb{F}_q^n$, the evaluation procedure proceeds as follows.

Step 1: Obtain $f(\mathbf{0})$ from $A_0[1]$, which was computed during the initialization phase.

Step 2: Enumerate $(x_n,\ldots,x_1) \in P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1}$ in ascending lexicographic order, starting from the second element. For each $(x_n,\ldots,x_1)$, compute $\mathrm{Sup}(\boldsymbol{x})$ using Algorithm 3, i.e.,

$$(k_h,\ldots,k_1) = \mathrm{Sup}(x_n,\ldots,x_1),$$

and

$$H_\epsilon = \mathrm{Tsum}_{k_\epsilon}(\boldsymbol{x}) \text{ for } \epsilon \in \{h-1, h-2,\ldots,1\}, \qquad H = \mathrm{Tsum}_{k_h}(\boldsymbol{x}).$$

Step 3: Output $f(\boldsymbol{x}) = \mathtt{EVA}(H, H_\epsilon, d, k_h,\ldots,k_1)$ using Algorithm 2.

As a proof of concept, we provide implementations of our algorithm for general d as well as a specialized version for $d = 2$, available at GitHub repository.

Complexity of Algorithm 3: Since there are in total $\left|P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1}\right| - 1$ nonzero elements, Lines (8), (9), and (16) are executed $\left|P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1}\right| - 1$ times. Each execution of these lines involves a single arithmetic operation. Furthermore, in Line 10, the support $(k_h,\ldots,k_1)$ of the current vector is determined, and the required indices $k_{i_1} - k_j + 1$ and

$k_{i_2} - k_j + 1$ for $i_1, i_2, j \in [0, h]$ are computed. This step requires at most $4d$ operations. Consequently, the overall time complexity of Algorithm 3 can be estimated as

$$3 \cdot \left|P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}\right| + 4d \cdot \left|P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}\right| = \mathcal{O}\left(d \cdot \left|P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}\right|\right).$$

C. Overall Complexity Analysis

We analyze the overall memory and time complexity of the proposed algorithm, depending on the method employed in the initialization phase.

Memory complexity: During the evaluation phase, no additional memory is allocated beyond the arrays constructed in the initialization phase. Hence, the overall memory complexity of the algorithm coincides with the memory complexity of the chosen initialization method.

Time complexity: The total running time consists of the initialization phase followed by the evaluation phase. In the evaluation phase, computing $f(\boldsymbol{x})$ for a given $\boldsymbol{x}$ requires at most d operations. This follows from Algorithm 2, where the inner loop over δ (line 7) is executed only when $\eta < d$ and $x_{k_\epsilon} - 1 + \eta < d$, and in that case performs at most d iterations. Therefore, evaluating the polynomial over the structured subset $P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$ requires

$$d \cdot \left|P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}\right| = d \cdot \prod_{j=1}^{s} \sum_{i=0}^{w_j} \binom{n_j}{i} (q-1)^i$$

operations.

Combining the costs of both phases, the overall time complexity of the algorithm is

$$\begin{cases} \mathcal{O}\left(d \cdot \binom{2n+d}{d}\right) + d \cdot \prod_{j=1}^{s} \sum_{i=0}^{w_j} \binom{n_j}{i} (q-1)^i, & \text{(with Method 1)}, \\ \mathcal{O}\left(\binom{n+d}{d}^2\right) + d \cdot \prod_{j=1}^{s} \sum_{i=0}^{w_j} \binom{n_j}{i} (q-1)^i, & \text{(with Method 2)}. \end{cases}$$

When $s = 1$ and $w_s = n$, the structured subset reduces to the complete space $\mathbb{F}_q^n$. In this case, the time complexity becomes $\mathcal{O}\left(d \cdot \binom{2n+d}{d}\right) + d \cdot q^n$ when using Method 1, and $\mathcal{O}\left(\binom{n+d}{d}^2\right) + d \cdot q^n$ when using Method 2.

As summarized in Table I, for evaluating a polynomial of degree d over $\mathbb{F}_q^n$, there exist specific parameter pairs, namely $(n = 1, d \geq 2)$, $(n = 2, d \geq 3)$, $(n = 3, d \geq 11)$, and $(n = 4, d \geq 53)$, for which using Method 2 in the initialization phase allows our algorithm to use less memory than FEA [10], while achieving the same time complexity. For all other pairs (n, d), employing Method 1 makes our algorithm more efficient than FEA in both time and memory. Moreover, the main advantage of our algorithm lies in its applicability to a more general structured input subset $P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$.

TABLE I: Comparison between different algorithms for the evaluation of a degree-d polynomial in n variables on all inputs over $\mathbb{F}_q^n$

Method	Time (T)	Memory
fast enumeration algorithm [10]	$\mathcal{O}\left(\binom{n+d}{d}^2 + d \cdot q^n\right)$	$\mathcal{O}\left(\log(q \cdot n) \cdot n \cdot \binom{n+d}{d}\right)$
our algorithm (with Method 1)	$\mathcal{O}\left(d \cdot \binom{2n+d}{d} + d \cdot q^n\right)$	$\mathcal{O}\left(\log q \cdot \binom{n+d}{d} + \log q \cdot d^2\right)$
our algorithm (with Method 2)	$\mathcal{O}\left(\binom{n+d}{d}^2 + d \cdot q^n\right)$	$\mathcal{O}\left(\log q \cdot \binom{n+d}{d}\right)$

V. Polynomial Evaluation over Structured Subsets of $\mathbb{F}_q^n$

Based on the sufficient condition given in Theorem 2, designing an efficient algorithm to evaluate a degree d polynomial $f(x_n, \ldots, x_1)$ over $P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$ reduces to specifying an evaluation order and an evaluation rule that satisfy Condition (8). In this section, we adopt the evaluation order and evaluation rule proposed in the Furue–Takagi fast enumeration algorithm (FEA) [10], and show that they can be directly reused for polynomial evaluation over $P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$. We further prove that these choices satisfy the sufficient condition of Theorem 2 with $S_0 = P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$.

Algorithm 2 Evaluation of $f(x)$ with $(k_h, \ldots, k_1) = \mathrm{Sup}(\boldsymbol{x})$, $H_\epsilon = \mathrm{Tsum}_{k_\epsilon}(\boldsymbol{x})$ for $\epsilon \in \{h, h-1, \cdots, 1\}$, where $H = H_h$ and $H_0 = 0$

procedure EVA($H, H_\epsilon, d, k_h, \ldots, k_1$)
  if H=1 **then**
    $A_0[k_1+1] = A_0[1] + A_{k_1}[0]$
  else
    for $\epsilon = h, h-1, \ldots, 1$ **do**
      $\eta \leftarrow H_{\epsilon-1}$
      for $\delta = \min(x_{k_\epsilon} - 1,\ d-1-\eta),\ \min(x_{k_\epsilon} - 1,\ d-1-\eta) - 1,\ \ldots,\ 0$ **do**
        $i \leftarrow \eta + \delta$
        if $i = H-1$ **then**
          if $x_{k_\epsilon} > 1$ **then** ▷ $\delta > 0$
            $A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[0] + A_{k_0^{x_{k_0}},\ldots,k_\epsilon^{\delta+1}}[0]$
          else ▷ $\delta = 0$
            $A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[k_\epsilon - k_{\epsilon-1} + 1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[0] + A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}} k_\epsilon}[0]$
          end if
        else if $i \le H-2$ **then**
          $num = 0$ **if** $i = d-1$ **else** 1
          if $\delta > 0$ **then**
            $\zeta \leftarrow x_{k_\epsilon} - \delta$
            if $\zeta > 1$ **then**
              $A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[1] + A_{k_0^{x_{k_0}},\ldots,k_\epsilon^{\delta+1}}[1 \times num]$
            else
              $A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_\epsilon^\delta}[k_{\epsilon+1} - k_\epsilon + 1] + A_{k_0^{x_{k_0}},\ldots,k_\epsilon^{\delta+1}}[(k_{\epsilon+1} - k_\epsilon + 1) \times num]$
            end if
          else
            $\zeta \leftarrow x_{k_\epsilon}$
            if $\zeta > 1$ **then**
              $A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[k_\epsilon - k_{\epsilon-1} + 1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[k_\epsilon - k_{\epsilon-1} + 1] + A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}} k_\epsilon}[1 \times num]$
            else
              $A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[k_\epsilon - k_{\epsilon-1} + 1] \leftarrow A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}}}[k_{\epsilon+1} - k_{\epsilon-1} + 1] + A_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}} k_\epsilon}[(k_{\epsilon+1} - k_\epsilon + 1) \times num]$
            end if
          end if
        end if
      end for
    end for
  end if
  return $A_0[k_1+1]$
end procedure

Evaluation Order and Evaluation Rule: Let $S_0 = P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1}$ be the structured input subset. We impose a lexicographic order on S_0, i.e.,

$$\forall j \in [1, |S_0| - 1]: \quad S_0[j-1] <_{\mathrm{lex}} S_0[j].$$

For any polynomial of the form $D_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^\delta}(f)$, its associated ordered input set $S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^\delta}(f)$ inherits the same lexicographic order. The evaluation rule used for all such polynomials is identical and is denoted by $\mathcal{R}_{\mathrm{lex}}$. The evaluation rule $\mathcal{R}_{\mathrm{lex}}$ is defined as follows. For any $j \ge 1$ and $l \ge 1$,

$$(x_n, \ldots, x_{j+1}, l-1, \underbrace{0, \ldots, 0}_{j-1}) \xrightarrow{\mathcal{R}_{\mathrm{lex}}} (x_n, \ldots, x_{j+1}, l, \underbrace{0, \ldots, 0}_{j-1}) \tag{13}$$

Equivalently, to evaluate a polynomial g at an input $x \in S$, we identify the smallest index j such that $x_j = l > 0$.

In this case, $\boldsymbol{x}$ can be written as $\boldsymbol{x} = (x_n, \ldots, x_{j+1}, l, \underbrace{0, \ldots, 0}_{j-1}) \in S$ and therefore $\boldsymbol{x}_{(j)} = (x_n, \ldots, x_{j+1}, l-1, \underbrace{0, \ldots, 0}_{j-1})$.

Under the evaluation rule $\mathcal{R}_{\text{lex}}$, the value of $g(\boldsymbol{x})$ is computed using the relation

$$g(\boldsymbol{x}) = g(\boldsymbol{x}_{(j)}) + D_j(g)(\boldsymbol{x}_{(j)}).$$

If $\boldsymbol{x}_{(j)} \in S$, the sufficient condition to apply the framework defined in Equation 7 over the structured subset is satisfied. This is formally stated in Lemma 4 in the next section.

A. Testing Sufficient Condition over Structured Subset

Lemma 4. *Let $S = P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1} = P_{n_s}^{w_s} \times \cdots \times P_{n_1}^{w_1} \subseteq \mathbb{F}_q^n$, $|S| > 1$ be an ordered set, imposed with lexicographic order. When evaluating a polynomial $g(x_n, \ldots, x_1)$ over S under the evaluation rule $\mathcal{R}_{lex}$ based on Equation 7, the following property holds*

$$\forall \boldsymbol{x} \in S \text{ and } \boldsymbol{x} \neq S[0]: \ \exists\, \boldsymbol{x}' \in S \text{ such that } \boldsymbol{x}' \text{ is the predecessor of } \boldsymbol{x} \text{ under } \mathcal{R}_{lex}. \tag{14}$$

Moreover, the input set for $D_i(g)$ is S_i, which is also an ordered set satisfying

$$S_i = P_{n'_m,\ldots,n'_1}^{w'_m,\ldots,w'_1} \subseteq S \subseteq \mathbb{F}_q^n,$$
$$\forall j \in [1, |S_i| - 1]: \ S_i[j-1] <_{lex} S_i[j],$$

for some integers $(w'_m, \ldots, w'_1, n'_m, \ldots, n'_1)$ where $w'_1 = 0$, $n'_1 = i - 1$ and $\sum_{j=1}^{m} n'_j = n$.

Algorithm 3 Enumerate $(k_h, \ldots, k_1)$ for inputs in $P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1}$

```
// H[0] stores Hw(x_{n_1}, ..., x_1), H[i] stores Hw(x_{sum_{j=1}^{i+1} n_j}, ..., x_{1+sum_{j=1}^{i} n_j}).
// K[h-i+1] stores k_i for 1 ≤ i ≤ h, i.e., (k_h, ..., k_1) = (K[1], ..., K[h]).
// num = I[i] represents that i ∈ [1 + sum_{j=1}^{num} n_j, sum_{j=1}^{num+1} n_j].
procedure ENU(K, H, I, W, N, h, end)
    for all i in [1, end] do
        num = I[i]
        for repeat in [1, q-1] do:
            K[h+1] = i
            H[num] = H[num] + 1 // increase the Hamming weight
            Output h+1 and the necessary (at most d+1) indices from K
            if H[num] < W[num] then
                ENU(K, H, I, W, N, h+1, K[h+1] - 1)
            else if H[num] = W[num] and num > 0 then
                ENU(K, H, I, W, N, h+1, N[num - 1])
            end if
            H[num] = H[num] - 1  //backtracking
        end for
    end for
end procedure

procedure MAIN
    W[] = {w_1, ..., w_s}, N[] = {n_1, n_1 + n_2, ..., sum_{i=1}^{s} n_i}
    K[n+1] = {n+1, 0, ..., 0}, H[s] = {0, ..., 0}
    Initialize I[]: I[0] = 0 and I[i] = j if N[j-1] < i ≤ N[j] for i ∈ [1, n]
    ENU(K, H, I, W, N, 0, n)
end procedure
```

Proof. For any $\boldsymbol{x} \in S$ and $\boldsymbol{x} \neq S[0] = \boldsymbol{0}^n$, it can always be written as follows:

$$\begin{cases} \exists k \in [1,n]: \ \boldsymbol{x} = (x_n,\ldots,x_1) = (\underbrace{x_n,\ldots,x_{1+n_k+\cdots+n_1}}_{\in P_{n_s}^{w_s}\times\cdots\times P_{n_{k+1}}^{w_{k+1}}}, \boldsymbol{v}^{n_k}, \underbrace{0,\ldots,0}_{\sum_{j=1}^{k-1} n_j}), \\ \exists i \in [1+\sum_{j=1}^{k-1} n_j, \sum_{j=1}^{k} n_j], l>0: \ \boldsymbol{v}^{n_k} = (x_{n_k+\cdots+n_1},\ldots,x_{i+1},l, \underbrace{0,\ldots,0}_{i-1-\sum_{j=1}^{k-1} n_j}) \in P_{n_k}^{w_k} \end{cases} \tag{15}$$

In other words, for any $\boldsymbol{x} \in S$ and $\boldsymbol{x} \neq S[0]$, there is always:

$$\exists i \in [1,n]: \ \boldsymbol{x} = (x_n,\ldots,x_{i+1},l,\underbrace{0,\ldots,0}_{i-1}). \tag{16}$$

Under the given evaluation rule $\mathcal{R}_{\text{lex}}$, to evaluate $g(\boldsymbol{x})$ where $\boldsymbol{x} \neq S[0]$ satisfies Equation 16, it is always possible to find a predecessor $\boldsymbol{x}'$ of $\boldsymbol{x}$, of the form

$$\boldsymbol{x}' = (x_n,\ldots,x_{i+1},l-1,\underbrace{0,\ldots,0}_{i-1}). \tag{17}$$

Since $\boldsymbol{x} \in S$, we only need to prove $\boldsymbol{x}' \in S$. From Equation 17 and Equation 16, it is clear that all entries, except i-th of $\boldsymbol{x}$ and $\boldsymbol{x}'$ are same. Therefore, using Equation 15, we have

$$\begin{aligned} \boldsymbol{u}^{n_k} &= (x_{n_k+\cdots+n_1},\ldots,x_{i+1},l-1, \underbrace{0,\ldots,0}_{i-1-\sum_{j=1}^{k-1} n_j}) \in P_{n_k}^{w_k}, \\ \boldsymbol{x}' &= (\underbrace{x_n,\ldots,x_{1+n_k+\cdots+n_1}}_{\in P_{n_s}^{w_s}\times\cdots\times P_{n_{k+1}}^{w_{k+1}}}, \boldsymbol{u}^{n_k}, \underbrace{0,\ldots,0}_{\sum_{j=1}^{k-1} n_j}) \in P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1} = S. \end{aligned}$$

Thus, to evaluate $g(\boldsymbol{x})$ where $\boldsymbol{x}$ satisfies Equation 16 based on Equation 7 under $\mathcal{R}_{\text{lex}}$, the following relation is used:

$$g(\boldsymbol{x}) = g(\boldsymbol{x}') + D_i(g)(\boldsymbol{x}').$$

Hence, $D_i(g)$ is used only for the following inputs to $g(x_n,\ldots,x_1)$:

$$\underbrace{(\underbrace{x_n,\ldots,x_{1+n_k+\cdots+n_1}}_{\in P_{n_s}^{w_s}\times\cdots\times P_{n_{k+1}}^{w_{k+1}}}, \underbrace{x_{n_k+\cdots+n_1},\ldots,x_{i+1},l, \underbrace{0,\ldots,0}_{i-1-\sum_{j=1}^{k-1} n_j}}_{\in P_{n_k}^{w_k}}, \underbrace{0,\ldots,0}_{\sum_{j=1}^{k-1} n_j}) \in S,$$

which are enumerated in ascending lex order when they are set as inputs to $g(x_n,\ldots,x_1)$. Due to the above proved property for a single element $\boldsymbol{x} \neq S[0]$ in S, the following elements will be set as inputs to $D_i(g)(x_n,\ldots,x_1)$ under $\mathcal{R}_{\text{lex}}$:

$$\underbrace{(\underbrace{x_n,\ldots,x_{1+n_k+\cdots+n_1}}_{\in P_{n_s}^{w_s}\times\cdots\times P_{n_{k+1}}^{w_{k+1}}}, \underbrace{x_{n_k+\cdots+n_1},\ldots,x_{i+1},l-1, \underbrace{0,\ldots,0}_{i-1-\sum_{j=1}^{k-1} n_j}}_{\in P_{n_k}^{w_k}}, \underbrace{0,\ldots,0}_{\sum_{j=1}^{k-1} n_j}) \in S, \tag{18}$$

which are also naturally enumerated in ascending lex order. In short, for $\boldsymbol{x}_1$, $\boldsymbol{x}_2$ in S satisfying $\boldsymbol{x}_1 <_{lex} \boldsymbol{x}_2$, the corresponding predecessors satisfy $\boldsymbol{x}_1' <_{lex} \boldsymbol{x}_2'$. This proves that the input set of $D_i(f)$ denoted by S_i is an ordered set, and

$$\forall j \in [1, |S_i|-1]: \ S_i[j-1] <_{\text{lex}} S_i[j].$$

Moreover, each element in S_i is of the form as in Equation 18. Hence, we have

$$\begin{aligned} \exists k \in [1,s]: \ S_i &= P_{n_s}^{w_s} \times \cdots \times P_{n_{k+1}}^{w_{k+1}} \times P_{n_k'}^{\min(n_k',w_k)} \times P_{i-1}^0, \\ n_k' &= n_k - (i-1-\sum_{j=1}^{k-1} n_j). \end{aligned}$$

Due to Equation 18, we also have $S_i \subseteq S$.

□

B. Application to Polynomial Evaluations over $P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$

Due to Lemma 4, when evaluating a polynomial $f(x_n,\ldots,x_1)$ of degree d over the ordered set $S_0 = P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$ under $\mathcal{R}_{\text{lex}}$ based on Equation 7, where

$$\forall j \in [1, |S_0|-1]: \ S_0[j-1] <_{\text{lex}} S_0[j],$$

the input set $S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}$ for any polynomial $D_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}(f)$ preserves the same structure as $P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$ with changes in the values of the parameters $(n_s,\ldots,n_1,w_s,\ldots,w_1)$, and the elements in $S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}$ are listed in ascending lex order. Moreover, according to Lemma 4, we have $S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}} \in P^{w'_q,\ldots,w'_2,0}_{n'_q,\ldots,n'_2,k_\epsilon-1}$, resulting in $k_1 < k_2 < \ldots < k_\epsilon$. This is formally stated in Corollary 1.

Corollary 1. *When evaluating a polynomial $f(x_n,\ldots,x_1)$ of degree d over the ordered set $S_0 = P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$ under $\mathcal{R}_{lex}$ based on Equation 7, where*

$$\forall j \in [1, |S_0|-1]: \ S_0[j-1] <_{lex} S_0[j],$$

there are in total $\binom{n+d}{d}$ possible polynomials $D_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}(f)$ appearing where

$$k_0 = 0, 1 \le k_1 < \ldots < k_i \le n, \sum_{i=1}^{\epsilon-1} x_{k_i} + \delta + 1 \le d.$$

Moreover, the input set of $D_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}(f)$ denoted by $S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}$ satisfies

$$\exists (n'_q,\ldots,n'_2,w'_q,\ldots,w'_2) \in \mathbb{N}^{2(q-1)}: \quad S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}} = P^{w'_q,\ldots,w'_2,0}_{n'_q,\ldots,n'_2,k_\epsilon-1} \subseteq \mathbb{F}_q^n,$$
$$\forall j \in [1, |S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}| - 1]: \quad S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}[j-1] <_{lex} S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}[j].$$

Therefore, using the evaluation order as the lexicographic order and the evaluation rule as $\mathcal{R}_{\text{lex}}$, we can safely apply the rephrased fast enumeration framework developed in Section III for polynomial evaluations over $P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}$.

Time complexity of the evaluation phase: For the algorithm given in the proof of Theorem 2, the time complexity of its evaluation phase is upper bounded by $d \cdot |P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}|$.

Time complexity of initialization phase: By Corollary 1, we have

$$S_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}[0] = \mathbf{0}^n.$$

Consequently, in the initialization phase, we only need to evaluate the corresponding values of $D_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}(f)(0^n)$. As discussed in Section IV-A, there are $\sum_{i=0}^{d}\binom{n+i-1}{i} = \binom{n+d}{d}$ possible polynomials $D_{k_0^{x_{k_0}},\ldots,k_{\epsilon-1}^{x_{k_{\epsilon-1}}},k_\epsilon^{\delta+1}}$, and the value of such polynomial at 0^n can be evaluated using Method 1 or Method 2 given in Section IV-A1 and IV-A2, respectively, and the corresponding time complexity of the initialization phase is given in Equations 10a and 11a.

Memory complexity: For the algorithm given in the proof of Theorem 2, we need $TAB_{\mathbf{ini}}$ of size $\log q \cdot \binom{n+d}{d}$. Also, in the evaluation phase, there are d steps to get the value of $f(\boldsymbol{x})$ and at each step we store the value of each polynomial in $TAB_{\mathbf{eva}}$ for further use. Therefore, the size of $TAB_{\mathbf{eva}}$ is $\log(q) \cdot d \cdot |P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}|$. Hence, the total memory complexity is at most $\log q \cdot \left(\binom{n+d}{d} + d \cdot |P^{w_s,\ldots,w_1}_{n_s,\ldots,n_1}|\right)$.

As mentioned in Section IV-B, the memory used to store the value of a polynomial at some input can be reused to store the value of the same polynomial at some different input. In other words, some values are never required for future evaluation and hence can be released. Using this strategy, the memory requirement can be further reduced.

To optimize memory complexity, we propose Lemma 5 to show that a large number of values of a polynomial can be released after evaluating the polynomial at a certain input $\boldsymbol{x}$.

Lemma 5. *After evaluating $f(\boldsymbol{x})$ for $\boldsymbol{x} = (x_n,\ldots,x_{i+1},\zeta,0,\ldots,0) \in S$ with $\zeta > 0$, the values $f(\boldsymbol{y})$ for all $\boldsymbol{y} = (y_n,\ldots,y_{i+1},l,0,\ldots,0) \in S$ with $l > 0$ and $\boldsymbol{y} <_{\text{lex}} \boldsymbol{x}$ are not required in any future evaluation step.*

Proof. Consider two sets

$$S_{<_{\text{lex}}\boldsymbol{x}} = \{\boldsymbol{y} \mid \boldsymbol{y} \in S, \boldsymbol{y} = (y_n,\ldots,y_{i+1},l,\underbrace{0,\ldots,0}_{i-1}) <_{\text{lex}} \boldsymbol{x}, \ l > 0\}$$

$$S_{\boldsymbol{x}<_{\text{lex}}} = \{\boldsymbol{y} \mid \boldsymbol{y} \in S, \boldsymbol{x} <_{\text{lex}} \boldsymbol{y} = (y_n,\ldots,y_{i+1},l,\underbrace{0,\ldots,0}_{i-1}), \ l > 0\}$$

To prove the lemma, we only need to show that

$$\forall \boldsymbol{z} \in S_{\boldsymbol{x}<_{\text{lex}}} : \text{ if } \boldsymbol{z}' \xrightarrow{\mathcal{R}_{\text{lex}}} \boldsymbol{z}, \text{ then } \boldsymbol{z}' \notin S_{<_{\text{lex}}\boldsymbol{x}}. \tag{19}$$

We prove Equation 19 case by case:

Case 1: When $\boldsymbol{z} = (z_n, \ldots, z_{j+1}, \zeta', \underbrace{0, \ldots, 0}_{j-1}) \in S_{\boldsymbol{x}<_{\text{lex}}}$ for $i \le j \le n$, according to the evaluation rule $\mathcal{R}_{\text{lex}}$, there exists a predecessor $\boldsymbol{z}'$ of $\boldsymbol{z}$ where

$$\boldsymbol{z}' = (z_n, \ldots, z_{j+1}, \zeta' - 1, \underbrace{0, \ldots, 0}_{j-1})$$

For $i < j$, we have $(z'_i, \ldots, z'_1)$, i.e., $z'_i = 0$ thus resulting in $\boldsymbol{z}' \notin S_{<_{\text{lex}}\boldsymbol{x}}$.
For $i = j$, from $\boldsymbol{x} <_{\text{lex}} \boldsymbol{z}$ we have $INT(\boldsymbol{z}) \ge INT(\boldsymbol{x}) + q^{i-1}$. Since $INT(\boldsymbol{z}) = INT(\boldsymbol{z}') + q^{i-1}$, it follows that $\boldsymbol{x} \le_{\text{lex}} \boldsymbol{z}'$, and hence $\boldsymbol{z}' \notin S_{<_{\text{lex}}\boldsymbol{x}}$.

Case 2: When $\boldsymbol{z} = (z_n, \ldots, z_{j+1}, \zeta', \underbrace{0, \ldots, 0}_{j-1}) \in S_{\boldsymbol{x}<_{\text{lex}}}$ for $1 \le j < i$, according to the evaluation rule $\mathcal{R}_{\text{lex}}$, there exists a predecessor $\boldsymbol{z}'$ of $\boldsymbol{z}$ where

$$\boldsymbol{z}' = (z_n, \ldots, z_{j+1}, \zeta' - 1, \underbrace{0, \ldots, 0}_{j-1})$$

If $(z'_i, \ldots, z'_1) \ne (l, \underbrace{0, \ldots, 0}_{i-1})$ for $l > 0$, we directly have $\boldsymbol{z}' \notin S_{<_{\text{lex}}\boldsymbol{x}}$.

If $(z'_i, \ldots, z'_1) = (l, \underbrace{0, \ldots, 0}_{i-1})$ for $l > 0$, since $j < i$, we have

$$\boldsymbol{z}' = (z_n, \ldots, z_{i+1}, l, \underbrace{0, \ldots, 0}_{i-1}), \ \boldsymbol{z} = (z_n, \ldots, z_{i+1}, l, \underbrace{0, \ldots, 0}_{i-j-1}, \zeta' = 1, \underbrace{0, \ldots, 0}_{j-1}).$$

Since $\boldsymbol{x} = (x_n, \ldots, x_{i+1}, \zeta, \underbrace{0, \ldots, 0}_{i-1}) <_{\text{lex}} \boldsymbol{z}$, we have $\boldsymbol{x} \le_{\text{lex}} \boldsymbol{z}'$. Therefore, we also have $\boldsymbol{z}' \notin S_{<_{\text{lex}}\boldsymbol{x}}$.

□

Consequently, by Lemma 5, the memory used to store $f(\mathbf{0}^{n-i}, 1, \mathbf{0}^{i-1})$ can be reused consecutively to store $f(x_n, \cdots, x_{i+1}, l, \mathbf{0}^{i-1})$, where $l > 0$ and $(x_n, \ldots, x_{i+1}, l, \mathbf{0}^{i-1})$ are traversed in lexicographic order. In particular, for Type-i memory, where $i = \sum_{j=0}^{\epsilon-1} x_{k_j} + \delta + 1$ the array $A_{x_0^{k_0}, \ldots, k_\epsilon^{\delta+1}}$ is used to store $D_{x_0^{k_0}, \ldots, k_\epsilon^{\delta+1}} f(0, \cdots, 0, 1, \mathbf{0}^{j-1}, \mathbf{0}^{k_\epsilon - 1})$ for $0 < j \le n - k_\epsilon + 1$ and can be reused throughout the evaluation process. Therefore, the required size of the array $A_{x_0^{k_0}, \ldots, k_\epsilon^{\delta+1}}$ is $n + 2 - k_\epsilon$. Hence, the overall memory requirement of the algorithm is

$$\begin{cases} \mathcal{O}\left(\log(q) \cdot \binom{n+d}{d} + \log(q) \cdot d^2\right), & \text{(with Method 1)}, \\ \mathcal{O}\left(\log(q) \cdot \binom{n+d}{d}\right), & \text{(with Method 2)}. \end{cases}$$

Remark 5 (Case of $q = p^r$)**.** *The case where $q = p^r$, with p prime and r a positive integer, has been previously considered in [10]. For completeness, we briefly recall the main idea and note that the same reduction applies to the proposed enumeration algorithm.*

Since $\mathbb{F}_{p^r}$ is an r-dimensional vector space over $\mathbb{F}_p$, let $\{\theta_1, \ldots, \theta_r\}$ be a basis of $\mathbb{F}_{p^r}$ over $\mathbb{F}_p$. Each variable $x_i \in \mathbb{F}_{p^r}$ can be expressed as

$$x_i = \sum_{j=1}^{r} x_j^{(i)} \theta_j, \quad x_j^{(i)} \in \mathbb{F}_p.$$

Under this representation, a polynomial in n variables over $\mathbb{F}_{p^r}$ can be viewed as r polynomials in nr variables over $\mathbb{F}_p$.

As shown in [10], this transformation enables the use of enumeration algorithms over $\mathbb{F}_{p^r}$. Applying the same reduction to our proposed algorithm yields an extension to the case $q = p^r$. Consequently, for $q = p^r$, the algorithm admits time complexity

$$\mathcal{O}\left(r \cdot \left(T_{\text{init}}(nr, d, p) + d\, p^{rn}\right)\right)$$

and memory complexity

$$\mathcal{O}\left(r \cdot M_{\text{init}}(nr, d, p)\right),$$

TABLE II: Example of our algorithm for evaluation of f over $\mathbb{F}_3^2$

(x_2, x_1)	$(k_h, \ldots, k_1)$	Update	$f(x_2, x_1)$
(0,0)	-	-	$A_0[1] = 1$
(0,1)	(1)	$A_0[2] = A_0[1] + A_{0,1}[0] = 1 + 1 = 2$	$A_0[2] = 2$
(0,2)	(1)	$A_{0,1}[1] = A_{0,1}[0] + A_{0,1,1}[0] = 1 + 2 = 0$ $A_0[2] = A_0[2] + A_{0,1}[1] = 2 + 0 = 2$	$A_0[2] = 2$
(1,0)	(2)	$A_0[3] = A_0[1] + A_{0,2}[0] = 1 + 1 = 2$	$A_0[3] = 2$
(1,1)	(2,1)	$A_{0,1}[2] = A_{0,1}[0] + A_{0,1,2}[0] = 1 + 2 = 0$ $A_0[2] = A_0[3] + A_{0,1}[2] = 2 + 0 = 2$	$A_0[2] = 2$
(1,2)	(2,1)	$A_{0,1}[1] = A_{0,1}[2] + A_{0,1,1}[0] = 0 + 2 = 2$ $A_0[2] = A_0[2] + A_{0,1}[1] = 2 + 2 = 1$	$A_0[2] = 1$
(2,0)	(2)	$A_{0,2}[1] = A_{0,2}[0] + A_{0,2,2}[0] = 1 + 0 = 1$ $A_0[3] = A_0[3] + A_{0,2}[1] = 2 + 1 = 0$	$A_0[3] = 0$
(2,1)	(2,1)	$A_{0,1}[2] = A_{0,1}[2] + A_{0,1,2}[0] = 0 + 2 = 2$ $A_0[2] = A_0[3] + A_{0,1}[2] = 0 + 2 = 2$	$A_0[2] = 2$
(2,2)	(2,1)	$A_{0,1}[1] = A_{0,1}[2] + A_{0,1,1}[0] = 2 + 2 = 1$ $A_0[2] = A_0[2] + A_{0,1}[1] = 2 + 1 = 0$	$A_0[2] = 0$

where $T_{\text{init}}(nr, d, p)$ and $M_{\text{init}}(nr, d, p)$ denote the initialization time and memory bounds corresponding to either Method 1 or Method 2 as derived in Section IV-A.

VI. Example of the Proposed Algorithm

We illustrate the behavior of the proposed algorithm through two concrete examples: one over the full space $\mathbb{F}_q^n$ and another over a structured subset $P_{n_s,\ldots,n_1}^{w_s,\ldots,w_1}$ of $\mathbb{F}_q^n$.

a) Evaluation over the full space.: Consider a quadratic polynomial in two variables x_1, x_2 over $\mathbb{F}_3$:

$$f(x_2, x_1) = x_1^2 + 2x_1x_2 + x_2 + 1.$$

In the initialization phase, we have

$$A_0[1] = 1, \quad A_{0,1}[0] = 1, \quad A_{0,2}[0] = 1, \quad A_{0,1,1}[0] = 2, \quad A_{0,1,2}[0] = 2, \quad A_{0,2,2}[0] = 0.$$

Table II presents the evaluation phase of the polynomial f over the full space $\mathbb{F}_3^2$, where the function is evaluated at each input using the proposed algorithm.

b) Evaluation over the structured subset: Consider a quadratic polynomial in three variables x_1, x_2, x_3 over $P_2^1 \times P_1^1 \subset \mathbb{F}_3^3$:

$$g(x_3, x_2, x_1) = 2x_3^2 + x_3x_1 + 2x_1x_2 + x_1^2 + x_2 + 2.$$

In the initialization phase, we have

$$\begin{aligned} &A_0[1] = 2, \quad A_{0,1}[0] = 1, \quad A_{0,2}[0] = 1, \quad A_{0,3}[0] = 2, \\ &A_{0,1,1}[0] = 2, \quad A_{0,1,2}[0] = 2, \quad A_{0,1,3}[0] = 1, \\ &A_{0,2,2}[0] = 0, \quad A_{0,2,3}[0] = 0, \quad A_{0,3,3}[0] = 1. \end{aligned}$$

Table III illustrates the evaluation phase of the polynomial g over the structured subset $P_2^1 \times P_1^1$, where g is evaluated at each $\boldsymbol{x} \in P_2^1 \times P_1^1$ using the proposed algorithm.

VII. Conclusion and Future Work

In this paper, we studied the problem of efficiently evaluating multivariate polynomials over structured subsets of $\mathbb{F}_q^n$. Our approach generalizes existing results for structured subsets of $\mathbb{F}_2^n$ [13] and subsumes previous fast enumeration algorithms over the full finite-field space [10]. It enables, for the first time, efficient exhaustive evaluation over structured finite-field domains such as Hamming-weight–constrained spaces, which naturally arise in cryptanalytic problems, including syndrome decoding and post-quantum signature schemes such as WAVE.

Several directions for future work remain open. These include optimizing the initialization phase and refining memory usage for large field sizes. More broadly, the abstraction developed in this work will serve as a useful

TABLE III: Example of our algorithm for polynomial evaluations in three variables over the structured subset $P_2^1 \times P_1^1 \subset \mathbb{F}_3^3$.

(x_3, x_2, x_1)	$(k_h, \ldots, k_1)$	Update	$g(x_3, x_2, x_1)$
(0,0,0)	–	–	$A_0[1] = 2$
(0,0,1)	(1)	$A_0[2] = A_0[1] + A_{0,1}[0] = 2 + 1 = 0$	$A_0[2] = 0$
(0,0,2)	(1)	$A_{0,1}[1] = A_{0,1}[0] + A_{0,1,1}[0] = 1 + 2 = 0$ $A_0[2] = A_0[2] + A_{0,1}[1] = 0 + 0 = 0$	$A_0[2] = 0$
(0,1,0)	(2)	$A_0[3] = A_0[1] + A_{0,2}[0] = 2 + 1 = 0$	$A_0[3] = 0$
(0,1,1)	(2,1)	$A_{0,1}[2] = A_{0,1}[0] + A_{0,1,2}[0] = 1 + 2 = 0$ $A_0[2] = A_0[3] + A_{0,1}[2] = 0 + 0 = 0$	$A_0[2] = 0$
(0,1,2)	(2,1)	$A_{0,1}[1] = A_{0,1}[2] + A_{0,1,1}[0] = 0 + 2 = 2$ $A_0[2] = A_0[2] + A_{0,1}[1] = 0 + 2 = 2$	$A_0[2] = 2$
(0,2,0)	(2)	$A_{0,2}[1] = A_{0,2}[0] + A_{0,2,2}[0] = 1 + 0 = 1$ $A_0[3] = A_0[3] + A_{0,2}[1] = 0 + 1 = 1$	$A_0[3] = 1$
(0,2,1)	(2,1)	$A_{0,1}[2] = A_{0,1}[2] + A_{0,1,2}[0] = 0 + 2 = 2$ $A_0[2] = A_0[3] + A_{0,1}[2] = 1 + 2 = 0$	$A_0[2] = 0$
(0,2,2)	(2,1)	$A_{0,1}[1] = A_{0,1}[2] + A_{0,1,1}[0] = 2 + 2 = 1$ $A_0[2] = A_0[2] + A_{0,1}[1] = 0 + 1 = 1$	$A_0[2] = 1$
(1,0,0)	(3)	$A_0[4] = A_0[1] + A_{0,3}[0] = 2 + 2 = 1$	$A_0[4] = 1$
(1,0,1)	(3,1)	$A_{0,1}[3] = A_{0,1}[0] + A_{0,1,3}[0] = 1 + 1 = 2$ $A_0[2] = A_0[4] + A_{0,1}[3] = 1 + 2 = 0$	$A_0[2] = 0$
(1,0,2)	(3,1)	$A_{0,1}[1] = A_{0,1}[3] + A_{0,1,1}[0] = 2 + 2 = 1$ $A_0[2] = A_0[2] + A_{0,1}[1] = 0 + 1 = 1$	$A_0[2] = 1$
(2,0,0)	(3)	$A_{0,3}[1] = A_{0,3}[0] + A_{0,3,3}[0] = 2 + 1 = 0$ $A_0[4] = A_0[4] + A_{0,3}[1] = 1 + 0 = 1$	$A_0[4] = 1$
(2,0,1)	(3,1)	$A_{0,1}[3] = A_{0,1}[3] + A_{0,1,3}[0] = 2 + 1 = 0$ $A_0[2] = A_0[4] + A_{0,1}[3] = 1 + 0 = 1$	$A_0[2] = 1$
(2,0,2)	(3,1)	$A_{0,1}[1] = A_{0,1}[3] + A_{0,1,1}[0] = 0 + 2 = 2$ $A_0[2] = A_0[2] + A_{0,1}[1] = 1 + 2 = 0$	$A_0[2] = 0$

tool for designing and analyzing fast enumeration algorithms over structured domains in finite fields, and may stimulate further research at the intersection of algebraic computation and cryptanalysis.

APPENDIX

A. A tighter bound on the number of operations required in the initialization phase

As discussed in Section *Complexity Analysis of the Initialization Phase* for Method 1, if $m_k = 0$, then i_k must also be zero in order to contribute to the evaluation of $D_{\boldsymbol{m}} f(0^n)$. Conversely, if $i_k = 0$, then necessarily $m_k = 0$ since $0 \leq m_k \leq i_k$.

Consider a monomial $\boldsymbol{m}$ of the form $x_{s_1}^{m_1} \cdots x_{s_r}^{m_r}$, where $m_k > 0$ for all k and $r \leq \sum_{k=1}^{r} m_k = M \leq d$. The monomials that may contribute to $D_{\boldsymbol{m}} f(0^n)$ are given by

$$S_{\boldsymbol{m}} = \left\{ \prod_{k=1}^{r} x_{s_k}^{i_k} \ : \ i_k \geq m_k > 0, \ \sum_{k=1}^{r} i_k \leq d \right\}. \tag{20}$$

For a fixed value of M, computing $|S_{\boldsymbol{m}}|$ is equivalent to counting the number of vectors $(i_k)_{k=1}^{r}$ such that $(i_k - m_k) \geq 0$ and $\sum_{k=1}^{r} (i_k - m_k) \leq d - M$. Therefore, the maximum number of monomials contributing to $D_{\boldsymbol{m}} f(0^n)$ is

$$|S_{\boldsymbol{m}}| = \binom{r + d - M}{r}.$$

This quantity also corresponds to the number of addition operations required to compute the M-th order derivative of the polynomial f with respect to the r variables $x_{s_1}, \ldots, x_{s_r}$. Clearly, the value of $|S_{\boldsymbol{m}}|$ is independent of the individual values of the m_k. Hence, all $\binom{M-1}{r-1}$ monomials of the form $\boldsymbol{m} = x_{s_1}^{m_1} \cdots x_{s_r}^{m_r}$ with $m_k > 0$ for all k and $\sum_{k=1}^{r} m_k = M$ require at most $\binom{r+d-M}{r}$ addition operations.

Consequently, the total number of addition operations required for all necessary derivatives in the initialization phase involving r variables is bounded by

$$\sum_{M=r}^{d} \binom{M-1}{r-1} \binom{r+d-M}{r} = \binom{r+d}{r}.$$

Since the number of multiplications per monomial is at most $d+1$, the total number of arithmetic operations required in the initialization phase is bounded by

$$(d+1) \cdot \sum_{r=1}^{\min(n,d)} \binom{n}{r} \binom{d+r}{2r}.$$

References

[1] O. Billet and H. Gilbert, "Cryptanalysis of rainbow," in *Security and Cryptography for Networks, 5th International Conference, SCN 2006, Maiori, Italy, September 6-8, 2006, Proceedings*, ser. Lecture Notes in Computer Science, R. D. Prisco and M. Yung, Eds., vol. 4116. Springer, 2006, pp. 336–347. [Online]. Available: https://doi.org/10.1007/11832072_23

[2] W. Beullens, "Improved cryptanalysis of UOV and rainbow," *IACR Cryptol. ePrint Arch.*, p. 1343, 2020. [Online]. Available: https://eprint.iacr.org/2020/1343

[3] H. Furue and M. Kudo, "Polynomial XL: A variant of the XL algorithm using macaulay matrices over polynomial rings," *CoRR*, vol. abs/2112.05023, 2021. [Online]. Available: https://arxiv.org/abs/2112.05023

[4] A. Joux and V. Vitse, "A crossbred algorithm for solving boolean polynomial systems," in *Number-Theoretic Methods in Cryptology - First International Conference, NuTMiC 2017, Warsaw, Poland, September 11-13, 2017, Revised Selected Papers*, ser. Lecture Notes in Computer Science, J. Kaczorowski, J. Pieprzyk, and J. Pomykala, Eds., vol. 10737. Springer, 2017, pp. 3–21. [Online]. Available: https://doi.org/10.1007/978-3-319-76620-1_1

[5] C. Bouillaguet, H.-C. Chen, C.-M. Cheng, T. Chou, R. Niederhagen, A. Shamir, and B.-Y. Yang, "Fast exhaustive search for polynomial systems in $\mathbb{F}_2$," in *Cryptographic Hardware and Embedded Systems, CHES 2010*, S. Mangard and F.-X. Standaert, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 203–218.

[6] I. Dinur and A. Shamir, "An Improved Algebraic Attack on Hamsi-256," in *FSE*, ser. Lecture Notes in Computer Science, vol. 6733. Springer, 2011, pp. 88–106.

[7] I. Dinur, "Cryptanalytic applications of the polynomial method for solving multivariate equation systems over GF(2)," in *EUROCRYPT (1)*, ser. Lecture Notes in Computer Science, vol. 12696. Springer, 2021, pp. 374–403.

[8] C. Bouillaguet, "Algorithm 1052: Evaluating a boolean polynomial on all possible inputs," *ACM Trans. Math. Softw.*, vol. 50, no. 4, pp. 28:1–28:37, 2024.

[9] I. Dinur and A. Shamir, "Cube Attacks on Tweakable Black Box Polynomials," in *EUROCRYPT*, ser. Lecture Notes in Computer Science, vol. 5479. Springer, 2009, pp. 278–299.

[10] H. Furue and T. Takagi, "Fast enumeration algorithm for multivariate polynomials over general finite fields," in *Post-Quantum Cryptography*, T. Johansson and D. Smith-Tone, Eds. Cham: Springer Nature Switzerland, 2023, pp. 357–378.

[11] E. R. Berlekamp, R. J. McEliece, and H. C. A. van Tilborg, "On the inherent intractability of certain coding problems (corresp.)," *IEEE Trans. Inf. Theory*, vol. 24, no. 3, pp. 384–386, 1978. [Online]. Available: https://doi.org/10.1109/TIT.1978.1055873

[12] T. Debris-Alazard, N. Sendrier, and J. Tillich, "Wave: A new code-based signature scheme," *CoRR*, vol. abs/1810.07554, 2018. [Online]. Available: http://arxiv.org/abs/1810.07554

[13] F. Liu, V. Dixit, D. Yamamoto, W. Ogata, S. Sarkar, and W. Meier, "Efficient polynomial evaluation over structured space and application to polynomial method," Cryptology ePrint Archive, Paper 2026/040, 2026. [Online]. Available: https://eprint.iacr.org/2026/040

[14] A. Salagean, R. Winter, M. Mandache-Salagean, and R. C. Phan, "Higher order differentiation over finite fields with applications to generalising the cube attack," *Des. Codes Cryptogr.*, vol. 84, no. 3, pp. 425–449, 2017. [Online]. Available: https://doi.org/10.1007/s10623-016-0277-5